

CHƯƠNG 19. HUẤN LUYỆN VÀ TRIỂN KHAI MÔ HÌNH TENSORFLOW QUY MÔ LỚN

Khi bạn đã có một mô hình tuyệt vời có khả năng dự đoán đáng kinh ngạc, bạn sẽ làm gì với nó? Chà, bạn cần đưa nó vào sản xuất! Việc này có thể đơn giản như chạy mô hình trên một lô dữ liệu, và có lẽ viết một tập lệnh để chạy mô hình này mỗi đêm. Tuy nhiên, nó thường phức tạp hơn nhiều. Các phần khác nhau trong cơ sở hạ tầng của bạn có thể cần sử dụng mô hình này trên dữ liệu trực tiếp, trong trường hợp đó bạn có thể sẽ muốn đóng gói mô hình của mình trong một dịch vụ web: theo cách này, bất kỳ phần nào trong cơ sở hạ tầng của bạn đều có thể truy vấn mô hình bất cứ lúc nào bằng cách sử dụng một API REST đơn giản (hoặc một giao thức khác), như chúng ta đã thảo luận trong Chương 2.

Nhưng theo thời gian, bạn sẽ cần thường xuyên huấn luyện lại mô hình của mình trên dữ liệu mới và đẩy phiên bản cập nhật vào sản xuất. Bạn phải xử lý việc quản lý phiên bản mô hình, chuyển đổi một cách linh hoạt từ mô hình này sang mô hình tiếp theo, có thể quay lại mô hình trước đó trong trường hợp có vấn đề, và có thể chạy nhiều mô hình khác nhau song song để thực hiện các thử nghiệm A/B. Nếu sản phẩm của bạn trở nên thành công, dịch vụ của bạn có thể bắt đầu nhận được một số lượng lớn các truy vấn mỗi giây (QPS), và nó phải mở rộng để hỗ trợ tải. Một giải pháp tuyệt vời để mở rộng dịch vụ của bạn, như bạn sẽ thấy trong chương này, là sử dụng TF Serving, trên cơ sở hạ tầng phần cứng của riêng bạn hoặc thông qua một dịch vụ đám mây như Google Vertex AI. Nó sẽ đảm nhiệm việc phục vụ mô hình của bạn một cách hiệu quả, xử lý các chuyển đổi mô hình linh hoạt, và nhiều hơn nữa. Nếu bạn sử dụng nền tảng đám mây, bạn cũng sẽ nhận được nhiều tính năng bổ sung, chẳng hạn như các công cụ giám sát mạnh mẽ.

Hơn nữa, nếu bạn có nhiều dữ liệu huấn luyện và các mô hình yêu cầu tính toán chuyên sâu, thì thời gian huấn luyện có thể kéo dài một cách không thể chấp nhận được. Nếu sản phẩm của bạn cần thích ứng nhanh chóng với các thay đổi, thì thời gian huấn luyện dài có thể là một trở ngại lớn (ví dụ, hãy nghĩ đến một hệ thống đề xuất tin tức quảng bá tin tức từ tuần trước). Có lẽ quan trọng hơn, thời gian huấn luyện dài sẽ ngăn cản bạn thử nghiệm những ý tưởng mới. Trong học máy (cũng như trong nhiều lĩnh vực khác), rất khó để biết trước ý tưởng nào sẽ hiệu quả, vì vậy bạn nên thử càng nhiều càng tốt, càng nhanh càng tốt. Một cách để tăng tốc độ huấn luyện là sử dụng các bộ tăng tốc phần cứng như GPU hoặc TPU. Để đi nhanh hơn nữa, bạn có thể huấn luyện một mô hình trên nhiều máy, mỗi máy được trang bị nhiều bộ tăng tốc phần cứng. API chiến lược phân phối đơn giản nhưng mạnh mẽ của TensorFlow giúp việc này trở nên dễ dàng, như bạn sẽ thấy.

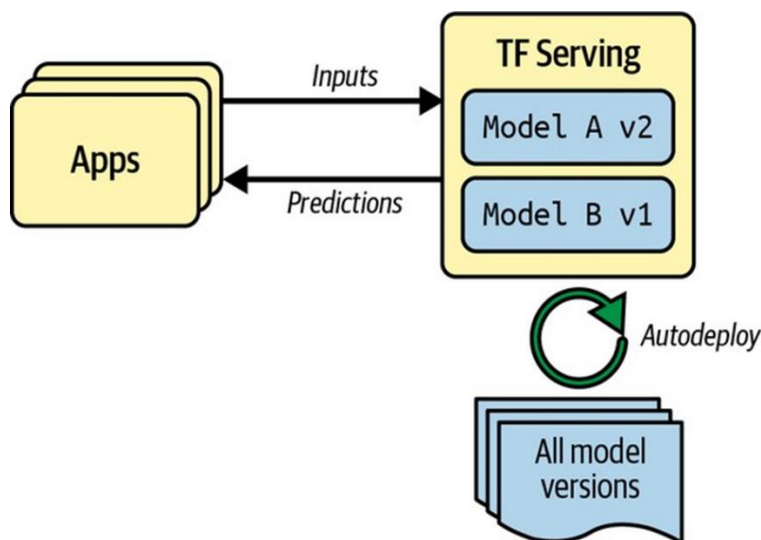
Trong chương này, chúng ta sẽ xem xét cách triển khai mô hình, đầu tiên sử dụng TF Serving, sau đó sử dụng Vertex AI. Chúng ta cũng sẽ xem nhanh về việc triển khai mô hình cho ứng dụng di động, thiết bị nhúng và ứng dụng web. Sau đó, chúng ta sẽ thảo luận về cách tăng tốc tính toán bằng GPU và cách huấn luyện mô hình trên nhiều thiết bị và máy chủ bằng API chiến lược phân phối. Cuối cùng, chúng ta sẽ khám phá cách huấn luyện mô hình và tinh chỉnh các siêu tham số của chúng ở quy mô lớn bằng Vertex AI. Có rất nhiều chủ đề để thảo luận, vậy hãy cùng tìm hiểu!

Phục vụ một mô hình TensorFlow

Khi bạn đã huấn luyện một mô hình TensorFlow, bạn có thể dễ dàng sử dụng nó trong bất kỳ mã Python nào: nếu đó là mô hình Keras, chỉ cần gọi phương thức `predict()` của nó! Nhưng khi cơ sở hạ tầng của bạn phát triển, sẽ đến lúc nên đóng gói mô hình của bạn trong một dịch vụ nhỏ mà vai trò duy nhất của nó là thực hiện các dự đoán và phần còn lại của cơ sở hạ tầng truy vấn nó (ví dụ: thông qua API REST hoặc gRPC). Điều này giúp tách mô hình của bạn khỏi phần còn lại của cơ sở hạ tầng, giúp dễ dàng chuyển đổi phiên bản mô hình hoặc mở rộng dịch vụ khi cần (độc lập với phần còn lại của cơ sở hạ tầng của bạn), thực hiện các thử nghiệm A/B và đảm bảo rằng tất cả các thành phần phần mềm của bạn đều dựa trên cùng một phiên bản mô hình. Nó cũng đơn giản hóa việc kiểm tra và phát triển, và nhiều hơn nữa. Bạn có thể tạo microservice của riêng mình bằng bất kỳ công nghệ nào bạn muốn (ví dụ: sử dụng thư viện Flask), nhưng tại sao phải phát minh lại bánh xe khi bạn có thể chỉ sử dụng TF Serving?

Sử dụng TensorFlow Serving

TF Serving là một máy chủ mô hình rất hiệu quả, đã được thử nghiệm kỹ lưỡng, được viết bằng C++. Nó có thể chịu được tải cao, phục vụ nhiều phiên bản mô hình của bạn và theo dõi kho lưu trữ mô hình để tự động triển khai các phiên bản mới nhất, v.v. (xem Hình 19-1).



Hình 19-1. TF Serving có thể phục vụ nhiều mô hình và tự động triển khai phiên bản mới nhất của mỗi mô hình.

Vậy giả sử bạn đã huấn luyện một mô hình MNIST bằng Keras và bạn muốn triển khai nó lên TF Serving. Điều đầu tiên bạn phải làm là xuất mô hình này sang định dạng SavedModel, đã được giới thiệu trong Chương 10.

Xuất SavedModels

Bạn đã biết cách lưu mô hình: chỉ cần gọi `model.save()`. Bây giờ để quản lý phiên bản mô hình, bạn chỉ cần tạo một thư mục con cho mỗi phiên bản mô hình. Dễ dàng!

```

from pathlib import Path
import tensorflow as tf

X_train, X_valid, X_test = [...] # tải và chia tập dữ liệu MNIST
model = [...] # xây dựng & huấn luyện một mô hình MNIST (cũng xử lý tiền xử lý ảnh)

model_name = "my_mnist_model"
model_version = "0001"
model_path = Path(model_name) / model_version
model.save(model_path, save_format="tf")

```

Thường thì nên bao gồm tất cả các lớp tiền xử lý trong mô hình cuối cùng mà bạn xuất để nó có thể tiếp nhận dữ liệu ở dạng tự nhiên sau khi được triển khai vào sản xuất. Điều này tránh việc phải xử lý tiền xử lý riêng biệt trong ứng dụng sử dụng mô hình. Việc đóng gói các bước tiền xử lý trong mô hình cũng giúp việc cập nhật chúng sau này đơn giản hơn và hạn chế rủi ro không khớp giữa mô hình và các bước tiền xử lý mà nó yêu cầu.

TensorFlow đi kèm với một giao diện dòng lệnh nhỏ `saved_model_cli` để kiểm tra SavedModels. Hãy sử dụng nó để kiểm tra mô hình đã xuất của chúng ta:

```
$ saved_model_cli show --dir my_mnist_model/0001
```

Kết quả này có nghĩa là gì? Chà, một SavedModel chứa một hoặc nhiều metagraph. Metagraph là một đồ thị tính toán cộng với một số định nghĩa chữ ký hàm, bao gồm tên đầu vào và đầu ra, kiểu và hình dạng của chúng. Mỗi metagraph được xác định bằng một tập hợp các thẻ. Ví dụ, bạn có thể muốn có một metagraph chứa toàn bộ đồ thị tính toán, bao gồm các hoạt động huấn luyện: bạn thường sẽ gắn thẻ này là “train”. Và bạn có thể có một metagraph khác chứa một đồ thị tính toán đã được cắt tỉa chỉ với các hoạt động dự đoán, bao gồm một số hoạt động cụ thể của GPU: thẻ này có thể được gắn là “serve”, “gpu”. Bạn cũng có thể muốn có các metagraph khác. Điều này có thể được thực hiện bằng cách sử dụng API SavedModel cấp thấp của TensorFlow. Tuy nhiên, khi bạn lưu một mô hình Keras bằng phương thức `save()` của nó, nó sẽ lưu một metagraph duy nhất được gắn thẻ là “serve”.

Hãy kiểm tra tập thẻ “serve” này:

```
$ saved_model_cli show --dir 0001/my_mnist_model --tag_set serve
```

The given SavedModel MetaGraphDef contains SignatureDefs with these keys:
 SignatureDef key: "saved_model_init_op"
 SignatureDef key: "serving_default"

Metagraph này chứa hai định nghĩa chữ ký: một hàm khởi tạo có tên “saved_model_init_op”, mà bạn không cần phải lo lắng, và một hàm phục vụ mặc định có tên “serving_default”. Khi lưu một mô hình Keras, hàm phục vụ mặc định là phương thức `call()` của mô hình, thực hiện các dự đoán, như bạn đã biết. Hãy lấy thêm chi tiết về hàm phục vụ này:

```
$ saved_model_cli show --dir 0001/my_mnist_model --tag_set serve \
--signature_def serving_default
```

The given SavedModel SignatureDef contains the following input(s):

inputs['flatten_input'] tensor_info:

dtype: DT_UINT8

shape: (-1, 28, 28)

name: serving_default_flatten_input:0

The given SavedModel SignatureDef contains the following output(s):

outputs['dense_1'] tensor_info:

dtype: DT_FLOAT

shape: (-1, 10)

name: StatefulPartitionedCall:0

Method name is: tensorflow/serving/predict

Lưu ý rằng đầu vào của hàm được đặt tên là “flatten_input”, và đầu ra được đặt tên là “dense_1”. Những cái này tương ứng với tên lớp đầu vào và đầu ra của mô hình Keras. Bạn cũng có thể thấy kiểu và hình dạng của dữ liệu đầu vào và đầu ra. Trông rất tốt! Bây giờ bạn đã có một SavedModel, bước tiếp theo là cài đặt TF Serving.

Cài đặt và khởi động TensorFlow Serving

Có nhiều cách để cài đặt TF Serving: sử dụng trình quản lý gói của hệ thống, sử dụng ảnh Docker, cài đặt từ mã nguồn, và nhiều hơn nữa. Vì Colab chạy trên Ubuntu, chúng ta có thể sử dụng trình quản lý gói apt của Ubuntu như sau:

```
url = "https://storage.googleapis.com/tensorflow-serving-apt"
src = "stable tensorflow-model-server tensorflow-model-server-universal"
!echo 'deb {url} {src}' > /etc/apt/sources.list.d/tensorflow-serving.list
!curl '{url}/tensorflow-serving.release.pub.gpg' | apt-key add -
!apt update -q && apt-get install -y tensorflow-model-server
```

```
%pip install -q -U tensorflow-serving-api
```

Mã này bắt đầu bằng cách thêm kho lưu trữ gói của TensorFlow vào danh sách các nguồn gói của Ubuntu. Sau đó, nó tải xuống khóa GPG công khai của TensorFlow và thêm nó vào danh sách khóa của trình quản lý gói để nó có thể xác minh chữ ký gói của TensorFlow. Tiếp theo, nó sử dụng apt để cài đặt gói tensorflow-model-server. Cuối cùng, nó cài đặt thư viện tensorflow-serving-api, mà chúng ta sẽ cần để giao tiếp với máy chủ.

Bây giờ chúng ta muốn khởi động máy chủ. Lệnh này sẽ yêu cầu đường dẫn tuyệt đối của thư mục mô hình cơ sở (nghĩa là đường dẫn đến my_mnist_model, không phải 0001), vì vậy hãy lưu nó vào biến môi trường MODEL_DIR:

```
import os
```

```
os.environ["MODEL_DIR"] = str(model_path.parent.absolute())
```

Sau đó, chúng ta có thể khởi động máy chủ:

```
%%bash --bg
tensorflow_model_server \
--port=8500 \
--rest_api_port=8501 \
--model_name=my_mnist_model \
--model_base_path="${MODEL_DIR}" >my_server.log 2>&1
```

Trong Jupyter hoặc Colab, lệnh ma thuật `%%bash --bg` thực thi ô dưới dạng tập lệnh bash, chạy nó trong nền. Phần `>my_server.log 2>&1` chuyển hướng đầu ra tiêu chuẩn và lỗi tiêu chuẩn đến tệp `my_server.log`. Và thế là xong! TF Serving hiện đang chạy trong nền, và nhật ký của nó được lưu vào `my_server.log`. Nó đã tải mô hình MNIST của chúng ta (phiên bản 1), và bây giờ nó đang chờ các yêu cầu gRPC và REST, tương ứng, trên các cổng 8500 và 8501.

CHẠY TF SERVING TRONG MỘT CONTAINER DOCKER Nếu bạn đang chạy sổ tay trên máy của mình và bạn đã cài đặt Docker, bạn có thể chạy `docker pull tensorflow/serving` trong một terminal để tải ảnh TF Serving. Đội ngũ TensorFlow rất khuyến nghị phương pháp cài đặt này vì nó đơn giản, sẽ không làm rối hệ thống của bạn và mang lại hiệu suất cao. Để khởi động máy chủ bên trong một container Docker, bạn có thể chạy lệnh sau trong một terminal:

```
$ docker run -it --rm -v "/path/to/my_mnist_model:/models/my_mnist_model" \
-p 8500:8500 -p 8501:8501 -e MODEL_NAME=my_mnist_model tensorflow/serving
```

Đây là ý nghĩa của tất cả các tùy chọn dòng lệnh này:

- `-it`: Làm cho container tương tác (để bạn có thể nhấn Ctrl-C để dừng nó) và hiển thị đầu ra của máy chủ.
- `--rm`: Xóa container khi bạn dừng nó: không cần làm lộn xộn máy của bạn với các container bị gián đoạn. Tuy nhiên, nó không xóa ảnh.
- `-v "/path/to/my_mnist_model:/models/my_mnist_model"`: Làm cho thư mục `my_mnist_model` của máy chủ có sẵn cho container tại đường dẫn `/models/mnist_model`. Bạn phải thay thế `/path/to/my_mnist_model` bằng đường dẫn tuyệt đối của thư mục này. Trên Windows, hãy nhớ sử dụng `\` thay vì `/` trong đường dẫn máy chủ, nhưng không phải trong đường dẫn container (vì container chạy trên Linux).
- `-p 8500:8500`: Khiến Docker engine chuyển tiếp cổng TCP 8500 của máy chủ đến cổng TCP 8500 của container. Theo mặc định, TF Serving sử dụng cổng này để phục vụ API gRPC.
- `-p 8501:8501`: Chuyển tiếp cổng TCP 8501 của máy chủ đến cổng TCP 8501 của container. Ảnh Docker được cấu hình để sử dụng cổng này theo mặc định để phục vụ API REST.
- `-e MODEL_NAME=my_mnist_model`: Đặt biến môi trường `MODEL_NAME` của container, để TF Serving biết mô hình nào sẽ phục vụ. Theo mặc định, nó sẽ tìm kiếm các mô hình trong thư mục `/models`, và nó sẽ tự động phục vụ phiên bản mới nhất mà nó tìm thấy.
- `tensorflow/serving`: Đây là tên của ảnh để chạy.

Bây giờ máy chủ đã hoạt động, hãy truy vấn nó, đầu tiên sử dụng API REST, sau đó là API gRPC.

Truy vấn TF Serving qua API REST

Hãy bắt đầu bằng cách tạo truy vấn. Nó phải chứa tên của chữ ký hàm bạn muốn gọi, và tất nhiên là dữ liệu đầu vào. Vì yêu cầu phải sử dụng định dạng JSON, chúng ta phải chuyển đổi các ảnh đầu vào từ một mảng NumPy sang một danh sách Python:

```
import json
import requests
# Giả định X_test đã được định nghĩa từ trước, ví dụ: X_test = tf.keras.datas
ets.mnist.load_data()[1][0]
X_new = X_test[:3] # giả vờ chúng ta có 3 ảnh chữ số mới để phân loại
request_json = json.dumps({
    "signature_name": "serving_default",
    "instances": X_new.tolist(),
})
```

Lưu ý rằng định dạng JSON hoàn toàn dựa trên văn bản. Chuỗi yêu cầu trông như thế này:

```
>>> request_json
'{"signature_name": "serving_default", "instances": [[[0, 0, 0, 0, ... ]]]}'
```

Bây giờ hãy gửi yêu cầu này đến TF Serving qua yêu cầu HTTP POST. Việc này có thể được thực hiện bằng thư viện requests (nó không phải là một phần của thư viện chuẩn Python, nhưng nó được cài đặt sẵn trên Colab):

```
server_url = "http://localhost:8501/v1/models/my_mnist_model:predict"
response = requests.post(server_url, data=request_json)
response.raise_for_status() # đưa ra một ngoại lệ trong trường hợp lỗi
response = response.json()
```

Phán đoán và viết công thức toán:

Trong đoạn văn bản này, không có công thức toán học tường minh nào được viết ra. Tuy nhiên, các khái niệm được đề cập ngụ ý đến các phép toán và cấu trúc dữ liệu cơ bản trong học máy, đặc biệt là liên quan đến các mô hình mạng nơ-ron:

1. **Mô hình dự đoán (`model.predict()`):** Đây là một hàm thực hiện phép toán tiến (forward pass) của mô hình. Nếu giả sử mô hình Keras là một mạng nơ-ron truyền thẳng (Feedforward Neural Network) với các lớp dày đặc (Dense layers), quá trình dự đoán cho một đầu vào x có thể được biểu diễn tổng quát như sau: $$$$$$y = f(x) $$$$ Trong đó:
 - x là dữ liệu đầu vào (ví dụ: ảnh MNIST 28x28 pixel).
 - f là hàm biến đổi phi tuyến tính tổng hợp của mô hình (chuỗi các lớp mạng nơ-ron).

- y là đầu ra dự đoán của mô hình (ví dụ: xác suất cho 10 chữ số trong MNIST). Đối với một lớp Dense trong mạng nơ-ron, phép toán cơ bản là biến đổi affine theo sau bởi một hàm kích hoạt:
$$a^{(l)} = W^{(l)} z^{(l)} + b^{(l)}$$
$$y^{(l)} = g(a^{(l)})$$
Trong đó:
- $a^{(l-1)}$ là đầu ra (kích hoạt) của lớp trước đó.
- $W^{(l)}$ là ma trận trọng số của lớp thứ l .
- $b^{(l)}$ là vector bias của lớp thứ l .
- $z^{(l)}$ là đầu vào tuyến tính cho hàm kích hoạt của lớp thứ l .
- $g^{(l)}$ là hàm kích hoạt của lớp thứ l (ví dụ: ReLU, Sigmoid, Softmax).
- $a^{(l)}$ là đầu ra (kích hoạt) của lớp thứ l .

2. Dữ liệu đầu vào và đầu ra (Input/Output tensor_info):

- `inputs['flatten_input'] tensor_info: dtype: DT_UINT8 shape: (-1, 28, 28)`
 - Đây là Tensor đầu vào có kiểu dữ liệu UINT8 (số nguyên không dấu 8 bit, thường dùng cho giá trị pixel 0-255).
 - Shape (-1, 28, 28) biểu thị:
 - -1: Kích thước batch động (số lượng mẫu trong một lô có thể thay đổi).
 - 28: Chiều cao của ảnh (28 pixel).
 - 28: Chiều rộng của ảnh (28 pixel).
 - Đối với ảnh MNIST, mỗi mẫu đầu vào là một ma trận 28x28.
- `outputs['dense_1'] tensor_info: dtype: DT_FLOAT shape: (-1, 10)`
 - Đây là Tensor đầu ra có kiểu dữ liệu FLOAT (số thực).
 - Shape (-1, 10) biểu thị:
 - -1: Kích thước batch động.
 - 10: Kích thước của vector đầu ra, thường là 10 lớp cho phân loại chữ số từ 0 đến 9. Nếu `dense_1` là lớp cuối cùng và dùng cho phân loại đa lớp, hàm kích hoạt thường là Softmax. Công thức của Softmax cho một vector đầu ra z có K phần tử là:
$$P(y=k|\mathbf{z}) = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}$$
Trong đó z_k là phần tử thứ k của vector đầu ra tuyến tính trước hàm Softmax.

Các công thức trên là những công thức toán học cơ bản mà một mô hình mạng nơ-ron, đặc biệt là mô hình Keras MNIST, sẽ sử dụng trong quá trình huấn luyện và dự đoán.

Truy vấn TF Serving qua API REST (Tiếp theo)

Nếu mọi việc suôn sẻ, phản hồi sẽ là một dictionary chứa một khóa “predictions” duy nhất. Giá trị tương ứng là danh sách các dự đoán. Danh sách này là một danh sách Python, vì vậy hãy chuyển đổi nó thành một mảng NumPy và làm tròn các số thập phân mà nó chứa đến hai chữ số thập phân:

```
>>> import numpy as np

>>> y_proba = np.array(response["predictions"])

>>> y_proba.round(2)
array([[0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 0.   , 1.   , 0.   , 0.   ],
       [0.   , 0.   , 0.99, 0.01, 0.   , 0.   , 0.   , 0.   , 0.   , 0.   ],
       [0.   , 0.97, 0.01, 0.   , 0.   , 0.   , 0.   , 0.01, 0.   , 0.   ]])
```

Hoan hô, chúng ta đã có các dự đoán! Mô hình tự tin gần 100% rằng ảnh đầu tiên là số 7, tự tin 99% rằng ảnh thứ hai là số 2, và tự tin 97% rằng ảnh thứ ba là số 1. Điều đó hoàn toàn chính xác.

API REST đẹp và đơn giản, và nó hoạt động tốt khi dữ liệu đầu vào và đầu ra không quá lớn. Hơn nữa, hầu hết mọi ứng dụng khách hàng đều có thể thực hiện các truy vấn REST mà không cần thêm phụ thuộc, trong khi các giao thức khác không phải lúc nào cũng sẵn có như vậy. Tuy nhiên, nó dựa trên JSON, là định dạng văn bản và khá dài dòng. Ví dụ, chúng ta phải chuyển đổi mảng NumPy sang danh sách Python, và mỗi số float cuối cùng được biểu diễn dưới dạng một chuỗi. Điều này rất không hiệu quả, cả về thời gian tuần tự hóa/giải tuần tự hóa — chúng ta phải chuyển đổi tất cả các số float sang chuỗi và ngược lại — và về kích thước tải trọng: nhiều số float cuối cùng được biểu diễn bằng hơn 15 ký tự, tương đương với hơn 120 bit cho số float 32 bit! Điều này sẽ dẫn đến độ trễ cao và sử dụng băng thông lớn khi truyền các mảng NumPy lớn. Vì vậy, hãy xem cách sử dụng gRPC thay thế.

Truy vấn TF Serving thông qua API gRPC

API gRPC mong đợi một protocol buffer PredictRequest đã được tuần tự hóa làm đầu vào, và nó xuất ra một protocol buffer PredictResponse đã được tuần tự hóa. Các protobuf này là một phần của thư viện tensorflow-serving-api, mà chúng ta đã cài đặt trước đó. Đầu tiên, hãy tạo yêu cầu:

```
from tensorflow_serving.apis.predict_pb2 import PredictRequest
import tensorflow as tf # Đảm bảo tensorflow đã được import
# Giả định model và X_new đã được định nghĩa từ trước
# Ví dụ: model = tf.keras.models.load_model(model_path)
# X_new = X_test[:3]

request = PredictRequest()
request.model_spec.name = "my_mnist_model" # Sử dụng model_name đã định nghĩa trước đó
request.model_spec.signature_name = "serving_default"
input_name = "flatten_input" # == model.input_names[0]
request.inputs[input_name].CopyFrom(tf.make_tensor_proto(X_new))
```

Đoạn mã này tạo ra một protocol buffer PredictRequest và điền vào các trường bắt buộc, bao gồm tên mô hình (đã được định nghĩa trước đó), tên chữ ký của hàm chúng ta muốn gọi, và cuối cùng là dữ liệu đầu vào, dưới dạng một protocol buffer Tensor. Hàm

`tf.make_tensor_proto()` tạo ra một protocol buffer Tensor dựa trên tensor hoặc mảng NumPy đã cho, trong trường hợp này là `X_new`.

Tiếp theo, chúng ta sẽ gửi yêu cầu đến máy chủ và nhận phản hồi. Để làm điều này, chúng ta sẽ cần thư viện `grpcio`, đã được cài đặt sẵn trong Colab:

```
import grpc
from tensorflow_serving.apis import prediction_service_pb2_grpc
# Giả định request đã được tạo ở trên

channel = grpc.insecure_channel('localhost:8500')
predict_service = prediction_service_pb2_grpc.PredictionServiceStub(channel)
response = predict_service.Predict(request, timeout=10.0)
```

Đoạn mã khá đơn giản: sau khi import, chúng ta tạo một kênh giao tiếp gRPC đến localhost trên cổng TCP 8500, sau đó chúng ta tạo một dịch vụ gRPC trên kênh này và sử dụng nó để gửi yêu cầu, với thời gian chờ là 10 giây. Lưu ý rằng cuộc gọi là đồng bộ: nó sẽ chặn cho đến khi nhận được phản hồi hoặc khi hết thời gian chờ. Trong ví dụ này, kênh không an toàn (không mã hóa, không xác thực), nhưng gRPC và TF Serving cũng hỗ trợ các kênh an toàn qua SSL/TLS.

Tiếp theo, hãy chuyển đổi protocol buffer `PredictResponse` thành một tensor:

```
# Giả định response đã nhận được từ gRPC call
output_name = "dense_1" # == model.output_names[0]
outputs_proto = response.outputs[output_name]
y_proba = tf.make_ndarray(outputs_proto)
```

Nếu bạn chạy đoạn mã này và in `y_proba.round(2)`, bạn sẽ nhận được chính xác các xác suất lớp ước tính giống như trước đó. Và thế là xong: chỉ với một vài dòng mã, bạn đã có thể truy cập mô hình TensorFlow của mình từ xa, sử dụng REST hoặc gRPC.

Triển khai phiên bản mô hình mới

Bây giờ hãy tạo một phiên bản mô hình mới và xuất một `SavedModel`, lần này vào thư mục `my_mnist_model/0002`:

```
# Giả định model_name đã được định nghĩa là "my_mnist_model"
# Giả định bạn đã có một mô hình MNIST mới được huấn luyện
# model = build_and_train_new_mnist_model() # ví dụ về hàm huấn luyện mô hình mới

model_version = "0002"
model_path = Path(model_name) / model_version
model.save(model_path, save_format="tf")
```

Theo các khoảng thời gian đều đặn (thời gian trễ có thể cấu hình được), TF Serving kiểm tra thư mục mô hình để tìm các phiên bản mô hình mới. Nếu tìm thấy một phiên bản, nó sẽ tự động xử lý quá trình chuyển đổi một cách linh hoạt: theo mặc định, nó trả lời các yêu cầu

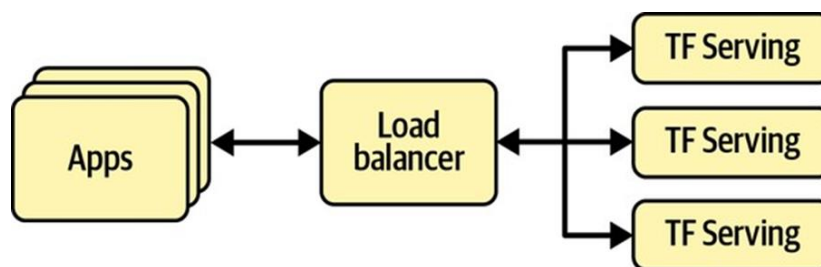
đang chờ xử lý (nếu có) bằng phiên bản mô hình trước đó, trong khi xử lý các yêu cầu mới bằng phiên bản mới. Ngay sau khi mọi yêu cầu đang chờ xử lý đã được trả lời, phiên bản mô hình trước đó sẽ được giải phóng. Bạn có thể thấy điều này hoạt động trong nhật ký TF Serving (trong `my_server.log`):

```
[...]
Reading SavedModel from: /models/my_mnist_model/0002
Reading meta graph with tags { serve }
[...]
Successfully loaded servable version {name: my_mnist_model version: 2}
Quiescing servable version {name: my_mnist_model version: 1}
Done quiescing servable version {name: my_mnist_model version: 1}
Unloading servable version {name: my_mnist_model version: 1}
```

Cách tiếp cận này mang lại sự chuyển đổi mượt mà, nhưng nó có thể sử dụng quá nhiều RAM — đặc biệt là RAM GPU, thường là bị hạn chế nhất. Trong trường hợp này, bạn có thể cấu hình TF Serving để nó xử lý tất cả các yêu cầu đang chờ xử lý bằng phiên bản mô hình trước đó và giải phóng nó trước khi tải và sử dụng phiên bản mô hình mới. Cấu hình này sẽ tránh việc có hai phiên bản mô hình được tải cùng lúc, nhưng dịch vụ sẽ không khả dụng trong một thời gian ngắn.

Như bạn có thể thấy, TF Serving giúp việc triển khai các mô hình mới trở nên đơn giản. Hơn nữa, nếu bạn phát hiện ra rằng phiên bản 2 không hoạt động tốt như bạn mong đợi, thì việc quay lại phiên bản 1 đơn giản như việc xóa thư mục `my_mnist_model/0002`.

Nếu bạn mong đợi nhận được nhiều truy vấn mỗi giây, bạn sẽ muốn triển khai TF Serving trên nhiều máy chủ và cân bằng tải các truy vấn (xem Hình 19-2). Điều này sẽ yêu cầu triển khai và quản lý nhiều container TF Serving trên các máy chủ này. Một cách để xử lý việc đó là sử dụng một công cụ như Kubernetes, là một hệ thống mã nguồn mở để đơn giản hóa việc điều phối container trên nhiều máy chủ. Nếu bạn không muốn mua, bảo trì và nâng cấp tất cả cơ sở hạ tầng phần cứng, bạn sẽ muốn sử dụng máy ảo trên nền tảng đám mây như Amazon AWS, Microsoft Azure, Google Cloud Platform, IBM Cloud, Alibaba Cloud, Oracle Cloud hoặc một số dịch vụ PaaS (Platform as a Service) khác. Việc quản lý tất cả các máy ảo, xử lý điều phối container (ngay cả với sự giúp đỡ của Kubernetes), chăm sóc cấu hình, điều chỉnh và giám sát TF Serving — tất cả những điều này có thể là một công việc toàn thời gian. May mắn thay, một số nhà cung cấp dịch vụ có thể lo tất cả những điều này cho bạn. Trong chương này, chúng ta sẽ sử dụng Vertex AI: đây là nền tảng duy nhất có TPU hiện nay; nó hỗ trợ TensorFlow 2, Scikit-Learn và XGBoost; và nó cung cấp một bộ dịch vụ AI tuyệt vời. Tuy nhiên, có một số nhà cung cấp khác trong lĩnh vực này cũng có khả năng phục vụ các mô hình TensorFlow, chẳng hạn như Amazon AWS SageMaker và Microsoft AI Platform, vì vậy hãy đảm bảo kiểm tra chúng.



Hình 19-2. Mở rộng TF Serving với cân bằng tải

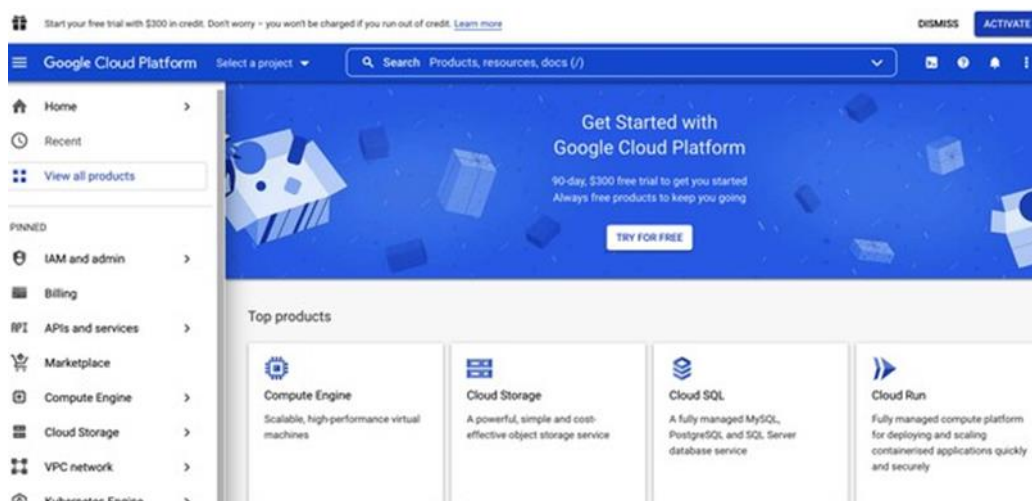
Bây giờ hãy xem cách phục vụ mô hình MNIST tuyệt vời của chúng ta trên đám mây!

Tạo dịch vụ dự đoán trên Vertex AI

Vertex AI là một nền tảng trong Google Cloud Platform (GCP) cung cấp một loạt các công cụ và dịch vụ liên quan đến AI. Bạn có thể tải lên tập dữ liệu, nhờ con người gán nhãn chúng, lưu trữ các tính năng thường được sử dụng trong một kho tính năng và sử dụng chúng để huấn luyện hoặc trong sản xuất, và huấn luyện mô hình trên nhiều máy chủ GPU hoặc TPU với điều chỉnh siêu tham số tự động hoặc tìm kiếm kiến trúc mô hình (AutoML). Bạn cũng có thể quản lý các mô hình đã huấn luyện của mình, sử dụng chúng để thực hiện dự đoán hàng loạt trên lượng lớn dữ liệu, lên lịch nhiều công việc cho quy trình dữ liệu của bạn, phục vụ mô hình của bạn qua REST hoặc gRPC ở quy mô lớn, và thử nghiệm với dữ liệu và mô hình của bạn trong môi trường Jupyter được lưu trữ gọi là Workbench. Thậm chí còn có dịch vụ Matching Engine cho phép bạn so sánh các vector rất hiệu quả (tức là hàng xóm gần nhất xấp xỉ). GCP cũng bao gồm các dịch vụ AI khác, chẳng hạn như API cho thị giác máy tính, dịch thuật, chuyển giọng nói thành văn bản, và nhiều hơn nữa.

Trước khi chúng ta bắt đầu, có một chút thiết lập cần thực hiện:

3. Đăng nhập vào tài khoản Google của bạn, và sau đó truy cập bảng điều khiển Google Cloud Platform (xem Hình 19-3). Nếu bạn chưa có tài khoản Google, bạn sẽ phải tạo một tài khoản.
4. Nếu đây là lần đầu tiên bạn sử dụng GCP, bạn sẽ phải đọc và chấp nhận các điều khoản và điều kiện. Người dùng mới được cung cấp bản dùng thử miễn phí, bao gồm 300 đô la tín dụng GCP mà bạn có thể sử dụng trong 90 ngày (tính đến tháng 5 năm 2022). Bạn sẽ chỉ cần một phần nhỏ trong số đó để thanh toán cho các dịch vụ bạn sẽ sử dụng trong chương này. Khi đăng ký bản dùng thử miễn phí, bạn vẫn cần tạo hồ sơ thanh toán và nhập số thẻ tín dụng: nó được sử dụng cho mục đích xác minh — có lẽ để tránh những người sử dụng bản dùng thử miễn phí nhiều lần — nhưng bạn sẽ không bị tính phí cho 300 đô la đầu tiên, và sau đó bạn sẽ chỉ bị tính phí nếu bạn chọn nâng cấp lên tài khoản trả phí.



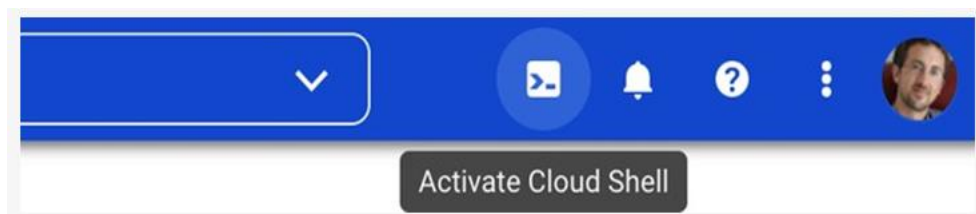
Hình 19-3. Bảng điều khiển Google Cloud Platform

3. Nếu bạn đã sử dụng GCP trước đây và bản dùng thử miễn phí của bạn đã hết hạn, thì các dịch vụ bạn sẽ sử dụng trong chương này sẽ tốn một ít tiền. Nó sẽ không quá nhiều, đặc biệt nếu bạn nhớ tắt các dịch vụ khi bạn không cần chúng nữa. Đảm bảo bạn hiểu và đồng ý với các điều kiện giá trước khi bạn chạy bất kỳ dịch vụ nào. Tôi từ chối mọi trách nhiệm nếu dịch vụ cuối cùng tốn kém hơn bạn mong đợi! Cũng đảm bảo tài khoản thanh toán của bạn đang hoạt động. Để kiểm tra, mở menu điều hướng ☰ ở trên cùng bên trái và nhấp vào “Billing”, sau đó đảm bảo bạn đã thiết lập phương thức thanh toán và tài khoản thanh toán đang hoạt động.
4. Mọi tài nguyên trong GCP đều thuộc về một dự án. Điều này bao gồm tất cả các máy ảo bạn có thể sử dụng, các tệp bạn lưu trữ và các công việc đào tạo bạn chạy. Khi bạn tạo tài khoản, GCP tự động tạo một dự án cho bạn, được gọi là “My First Project”. Nếu muốn, bạn có thể thay đổi tên hiển thị của nó bằng cách truy cập cài đặt dự án: trong menu điều hướng ☰, chọn “IAM and admin → Settings”, thay đổi tên hiển thị của dự án và nhấp vào SAVE. Lưu ý rằng dự án cũng có một ID và số duy nhất. Bạn có thể chọn ID dự án khi tạo dự án, nhưng bạn không thể thay đổi nó sau này. Số dự án được tự động tạo và không thể thay đổi. Nếu bạn muốn tạo một dự án mới, nhấp vào tên dự án ở đầu trang, sau đó nhấp vào NEW PROJECT và nhập tên dự án. Bạn cũng có thể nhấp vào EDIT để đặt ID dự án. Đảm bảo thanh toán đang hoạt động cho dự án mới này để phí dịch vụ có thể được thanh toán (vào tín dụng miễn phí của bạn, nếu có).
5. Bây giờ bạn đã có tài khoản GCP và một dự án, và thanh toán đã được kích hoạt, bạn phải kích hoạt các API bạn cần. Trong menu điều hướng ☰, chọn “APIs and services”, và đảm bảo Cloud Storage API được bật. Nếu cần, nhấp vào + ENABLE APIS AND SERVICES, tìm Cloud Storage và bật nó. Cũng bật Vertex AI API.

Bạn có thể tiếp tục thực hiện mọi thứ thông qua bảng điều khiển GCP, nhưng tôi khuyên bạn nên sử dụng Python thay thế: theo cách này, bạn có thể viết các tập lệnh để tự động hóa gần như mọi thứ bạn muốn với GCP, và nó thường tiện lợi hơn là nhấp qua các menu và biểu mẫu, đặc biệt đối với các tác vụ thông thường.

GOOGLE CLOUD CLI VÀ SHELL Giao diện dòng lệnh (CLI) của Google Cloud bao gồm lệnh `gcloud`, cho phép bạn kiểm soát hầu hết mọi thứ trong GCP, và `gsutil`, cho phép bạn tương tác với Google Cloud Storage. CLI này đã được cài đặt sẵn trong Colab: tất cả những gì bạn cần làm là xác thực bằng `google.auth.authenticate_user()`, và bạn đã sẵn sàng. Ví dụ, `!gcloud config list` sẽ hiển thị cấu hình.

GCP cũng cung cấp một môi trường shell được cấu hình sẵn gọi là Google Cloud Shell, mà bạn có thể sử dụng trực tiếp trong trình duyệt web của mình; nó chạy trên một máy ảo Linux (Debian) miễn phí với Google Cloud SDK đã được cài đặt sẵn và cấu hình cho bạn, vì vậy không cần xác thực. Cloud Shell có sẵn ở bất cứ đâu trong GCP: chỉ cần nhấp vào biểu tượng Activate Cloud Shell ở phía trên bên phải trang (xem Hình 19-4).



Hình 19-4. Kích hoạt Google Cloud Shell

Nếu bạn muốn cài đặt CLI trên máy của mình, thì sau khi cài đặt, bạn cần khởi tạo nó bằng cách chạy `gcloud init`: làm theo hướng dẫn để đăng nhập vào GCP và cấp quyền truy cập vào tài nguyên GCP của bạn, sau đó chọn dự án GCP mặc định bạn muốn sử dụng (nếu bạn có nhiều hơn một) và khu vực mặc định nơi bạn muốn các công việc của mình chạy.

Điều đầu tiên bạn cần làm trước khi có thể sử dụng bất kỳ dịch vụ GCP nào là xác thực. Giải pháp đơn giản nhất khi sử dụng Colab là thực thi đoạn mã sau:

```
from google.colab import auth
auth.authenticate_user()
```

Quá trình xác thực dựa trên OAuth 2.0: một cửa sổ bật lên sẽ yêu cầu bạn xác nhận rằng bạn muốn sở tay Colab truy cập thông tin đăng nhập Google của bạn. Nếu bạn chấp nhận, bạn phải chọn cùng tài khoản Google bạn đã sử dụng cho GCP. Sau đó, bạn sẽ được yêu cầu xác nhận rằng bạn đồng ý cấp cho Colab toàn quyền truy cập vào tất cả dữ liệu của bạn trên Google Drive và trong GCP. Nếu bạn cho phép truy cập, chỉ sổ tay hiện tại mới có quyền truy cập, và chỉ cho đến khi thời gian chạy Colab hết hạn. Rõ ràng, bạn chỉ nên chấp nhận điều này nếu bạn tin tưởng mã trong sổ tay.

Phán đoán và công thức toán:

Trong phần này của văn bản, không có công thức toán học tường minh nào xuất hiện. Tuy nhiên, các khái niệm liên quan đến “hiệu suất” và “kích thước tải trọng” khi so sánh REST và gRPC vẫn có thể được liên hệ với các phép đo định lượng:

5. Hiệu suất Serialization/Deserialization (REST vs gRPC):

- **REST (JSON):** Chuyển đổi số float thành chuỗi và ngược lại. Quá trình này có thể tốn thời gian và tài nguyên CPU. Ví dụ: số float 0.123456789 có thể được biểu diễn dưới dạng chuỗi “0.123456789”, chiếm nhiều byte hơn so với biểu diễn nhị phân.
- **gRPC (Protocol Buffers):** Sử dụng định dạng nhị phân nhỏ gọn hơn. Việc chuyển đổi từ dữ liệu gốc sang định dạng protobuf và ngược lại nhanh hơn và hiệu quả hơn về không gian.

6. Kích thước tải trọng (Payload Size):

- **REST (JSON):** Văn bản dài dòng, mỗi số float có thể chiếm hơn 15 ký tự (cho float 32-bit). Một float 32-bit (single-precision floating-point number) trong máy tính chiếm 4 byte = 32 bit. Nếu biểu diễn chuỗi của nó chiếm 15 ký tự, và mỗi ký tự là 1 byte (ASCII/UTF-8 đơn giản), thì sẽ tốn 15 byte. Tức là tăng kích thước gấp $15/4 = 3.75$ lần. Nếu “hơn 120 bit” được nhắc đến, có thể là $15 \text{ ký tự} \times 8 \text{ bit/ký tự} = 120 \text{ bit}$. Điều này dẫn đến công thức so sánh hiệu quả truyền tải: $\text{REST} \gg \text{gRPC}$. Sự khác biệt về kích thước này ảnh hưởng trực tiếp đến băng thông sử dụng và độ trễ truyền dữ liệu.

Mặc dù không có công thức toán học phức tạp nào, nhưng những thảo luận trên ngụ ý rằng việc tối ưu hóa hiệu suất (thời gian tính toán và băng thông) là rất quan trọng khi triển khai mô hình học máy ở quy mô lớn.

Xác thực và Cấp quyền trên GCP

Nói chung, việc sử dụng xác thực OAuth 2.0 chỉ được khuyến nghị khi một ứng dụng cần truy cập dữ liệu cá nhân hoặc tài nguyên của người dùng từ một ứng dụng khác, thay mặt người dùng. Ví dụ, một số ứng dụng cho phép người dùng lưu dữ liệu vào Google Drive của họ, nhưng để làm được điều đó, ứng dụng trước tiên cần người dùng xác thực với Google và cho phép truy cập vào Google Drive. Nói chung, ứng dụng sẽ chỉ yêu cầu mức độ truy cập mà nó cần; nó sẽ không phải là quyền truy cập không giới hạn: ví dụ, ứng dụng sẽ chỉ yêu cầu truy cập vào Google Drive, không phải Gmail hay bất kỳ dịch vụ Google nào khác. Hơn nữa, quyền ủy quyền thường hết hạn sau một thời gian, và nó luôn có thể bị thu hồi.

Khi một ứng dụng cần truy cập một dịch vụ trên GCP thay mặt cho chính nó, không phải thay mặt người dùng, thì nó thường nên sử dụng tài khoản dịch vụ (service account). Ví dụ, nếu bạn xây dựng một trang web cần gửi yêu cầu dự đoán đến một điểm cuối của Vertex AI, thì trang web đó sẽ truy cập dịch vụ thay mặt cho chính nó. Không có dữ liệu hoặc tài nguyên nào mà nó cần truy cập trong tài khoản Google của người dùng. Trên thực tế, nhiều người dùng của trang web thậm chí sẽ không có tài khoản Google. Đối với kịch bản này, bạn trước tiên cần tạo một tài khoản dịch vụ. Chọn “IAM and admin → Service accounts” trong menu điều hướng ☰ của bảng điều khiển GCP (hoặc sử dụng hộp tìm kiếm), sau đó nhấp vào + CREATE SERVICE ACCOUNT, điền vào trang đầu tiên của biểu mẫu (tên tài khoản dịch vụ, ID, mô tả), và nhấp vào CREATE AND CONTINUE. Tiếp theo, bạn phải cấp cho tài khoản này một số quyền truy cập. Chọn vai trò “Vertex AI user”: điều này sẽ cho phép tài khoản dịch vụ thực hiện dự đoán và sử dụng các dịch vụ Vertex AI khác, nhưng không có gì khác. Nhấp vào

CONTINUE. Bạn có thể tùy chọn cấp cho một số người dùng quyền truy cập vào tài khoản dịch vụ: điều này hữu ích khi tài khoản người dùng GCP của bạn là một phần của một tổ chức và bạn muốn ủy quyền cho những người dùng khác trong tổ chức triển khai các ứng dụng sẽ dựa trên tài khoản dịch vụ này, hoặc để quản lý tài khoản dịch vụ đó. Tiếp theo, nhấp vào DONE.

Khi bạn đã tạo một tài khoản dịch vụ, ứng dụng của bạn phải xác thực với tư cách là tài khoản dịch vụ đó. Có một số cách để làm điều đó. Nếu ứng dụng của bạn được lưu trữ trên GCP — ví dụ, nếu bạn đang viết mã một trang web được lưu trữ trên Google Compute Engine — thì giải pháp đơn giản nhất và an toàn nhất là đính kèm tài khoản dịch vụ vào tài nguyên GCP lưu trữ trang web của bạn, chẳng hạn như một phiên bản máy ảo (VM instance) hoặc dịch vụ Google App Engine. Điều này có thể được thực hiện khi tạo tài nguyên GCP, bằng cách chọn tài khoản dịch vụ trong phần “Identity and API access”. Một số tài nguyên, chẳng hạn như phiên bản máy ảo, cũng cho phép bạn đính kèm tài khoản dịch vụ sau khi phiên bản máy ảo được tạo: bạn phải dừng nó và chỉnh sửa cài đặt của nó. Trong mọi trường hợp, khi một tài khoản dịch vụ được đính kèm vào một phiên bản máy ảo, hoặc bất kỳ tài nguyên GCP nào khác đang chạy mã của bạn, các thư viện khách hàng của GCP (sẽ được thảo luận ngay sau đây) sẽ tự động xác thực với tư cách là tài khoản dịch vụ đã chọn, không cần thêm bước nào.

Nếu ứng dụng của bạn được lưu trữ bằng Kubernetes, thì bạn nên sử dụng dịch vụ Workload Identity của Google để ánh xạ tài khoản dịch vụ phù hợp với từng tài khoản dịch vụ Kubernetes. Nếu ứng dụng của bạn không được lưu trữ trên GCP — ví dụ, nếu bạn chỉ đang chạy sổ tay Jupyter trên máy của mình — thì bạn có thể sử dụng dịch vụ Workload Identity Federation (đây là tùy chọn an toàn nhất nhưng khó nhất), hoặc chỉ cần tạo khóa truy cập cho tài khoản dịch vụ của bạn, lưu nó vào một tệp JSON, và trở biến môi trường GOOGLE_APPLICATION_CREDENTIALS đến nó để ứng dụng khách của bạn có thể truy cập. Bạn có thể quản lý các khóa truy cập bằng cách nhấp vào tài khoản dịch vụ bạn vừa tạo, và sau đó mở tab KEYS. Đảm bảo giữ tệp khóa bí mật: nó giống như một mật khẩu cho tài khoản dịch vụ. Để biết thêm chi tiết về việc thiết lập xác thực và cấp quyền để ứng dụng của bạn có thể truy cập các dịch vụ GCP, hãy xem tài liệu.

Bây giờ hãy tạo một nhóm lưu trữ Google Cloud Storage (GCS bucket) để lưu trữ các SavedModels của chúng ta (một GCS bucket là một vùng chứa cho dữ liệu của bạn). Để làm điều này, chúng ta sẽ sử dụng thư viện google-cloud-storage, đã được cài đặt sẵn trong Colab. Chúng ta đầu tiên tạo một đối tượng Client, sẽ đóng vai trò là giao diện với GCS, sau đó chúng ta sử dụng nó để tạo nhóm:

```
from google.cloud import storage

project_id = "my_project" # thay đổi cái này thành ID dự án của bạn
bucket_name = "my_bucket" # thay đổi cái này thành một tên nhóm duy nhất
location = "us-central1"

storage_client = storage.Client(project=project_id)
bucket = storage_client.create_bucket(bucket_name, location=location)
```

GCS sử dụng một không gian tên duy nhất trên toàn thế giới cho các nhóm, vì vậy các tên đơn giản như “machine-learning” rất có thể sẽ không khả dụng. Đảm bảo tên nhóm tuân thủ các quy ước đặt tên DNS, vì nó có thể được sử dụng trong các bản ghi DNS. Hơn nữa, tên nhóm là công khai, vì vậy đừng đặt bất cứ điều gì riêng tư vào tên. Thường thì người ta sử dụng tên miền của bạn, tên công ty của bạn, hoặc ID dự án của bạn làm tiền tố để đảm bảo tính duy nhất, hoặc đơn giản là sử dụng một số ngẫu nhiên làm một phần của tên.

Bạn có thể thay đổi khu vực nếu muốn, nhưng hãy chắc chắn chọn một khu vực hỗ trợ GPU. Ngoài ra, bạn có thể muốn xem xét thực tế là giá cả thay đổi rất nhiều giữa các khu vực, một số khu vực sản xuất nhiều CO₂ hơn những khu vực khác, một số khu vực không hỗ trợ tất cả các dịch vụ, và việc sử dụng một nhóm một khu vực cải thiện hiệu suất. Xem danh sách các khu vực của Google Cloud và tài liệu của Vertex AI về các vị trí để biết thêm chi tiết. Nếu bạn không chắc chắn, tốt nhất có thể là gắn bó với “us-central1”.

Tiếp theo, hãy tải thư mục `my_mnist_model` lên nhóm mới. Các tệp trong GCS được gọi là blob (hoặc đối tượng), và về bản chất, tất cả chúng đều được đặt trong nhóm mà không có bất kỳ cấu trúc thư mục nào. Tên blob có thể là các chuỗi Unicode tùy ý, và chúng thậm chí có thể chứa dấu gạch chéo (/ forward slashes). Bảng điều khiển GCP và các công cụ khác sử dụng các dấu gạch chéo này để tạo ra ảo giác rằng có các thư mục. Vì vậy, khi chúng ta tải thư mục `my_mnist_model` lên, chúng ta chỉ quan tâm đến các tệp, không phải các thư mục:

```
from pathlib import Path
# Giả định bucket đã được tạo

def upload_directory(bucket, dirpath):
    dirpath = Path(dirpath)
    for filepath in dirpath.glob("**/*"):
        if filepath.is_file():
            # Tạo đường dẫn tương đối để blob_name không bao gồm thư mục gốc
            # của model
            blob_name = filepath.relative_to(dirpath.parent).as_posix()
            blob = bucket.blob(blob_name)
            blob.upload_from_filename(filepath)

# Thay thế "my_mnist_model" bằng đường dẫn thực tế đến thư mục mô hình của bạn
upload_directory(bucket, "my_mnist_model")
```

Hàm này hoạt động tốt bây giờ, nhưng nó sẽ rất chậm nếu có nhiều tệp để tải lên. Không quá khó để tăng tốc nó lên rất nhiều bằng cách đa luồng hóa nó (xem sổ tay để biết triển khai). Ngoài ra, nếu bạn có Google Cloud CLI, thì bạn có thể sử dụng lệnh sau thay thế:

```
!gsutil -m cp -r my_mnist_model gs://{bucket_name}/
```

Tiếp theo, hãy thông báo cho Vertex AI về mô hình MNIST của chúng ta. Để giao tiếp với Vertex AI, chúng ta có thể sử dụng thư viện `google-cloud-aiplatform` (nó vẫn sử dụng tên AI Platform cũ thay vì Vertex AI). Nó không được cài đặt sẵn trong Colab, vì vậy chúng ta cần cài đặt nó. Sau đó, chúng ta có thể import thư viện và khởi tạo nó — chỉ để chỉ định một số giá

trị mặc định cho ID dự án và vị trí — sau đó chúng ta có thể tạo một mô hình Vertex AI mới: chúng ta chỉ định tên hiển thị, đường dẫn GCS đến mô hình của chúng ta (trong trường hợp này là phiên bản 0001), và URL của container Docker chúng ta muốn Vertex AI sử dụng để chạy mô hình này. Nếu bạn truy cập URL đó và điều hướng lên một cấp, bạn sẽ tìm thấy các container khác mà bạn có thể sử dụng. Cái này hỗ trợ TensorFlow 2.8 với GPU:

```
# Cài đặt thư viện nếu chưa có
# %pip install google-cloud-aiplatform

from google.cloud import aiplatform

server_image = "gcr.io/cloud-aiplatform/prediction/tf2-gpu.2-8:latest"

aiplatform.init(project=project_id, location=location) # Sử dụng project_id và location đã định nghĩa ở trên
mnist_model = aiplatform.Model.upload(
    display_name="mnist",
    artifact_uri=f"gs://{bucket_name}/my_mnist_model/0001",
    serving_container_image_uri=server_image,
)
```

Bây giờ hãy triển khai mô hình này để chúng ta có thể truy vấn nó qua API gRPC hoặc REST để thực hiện dự đoán. Để làm điều này, trước tiên chúng ta cần tạo một điểm cuối (endpoint). Đây là nơi các ứng dụng khách hàng kết nối khi chúng muốn truy cập một dịch vụ. Sau đó, chúng ta cần triển khai mô hình của chúng ta đến điểm cuối này:

```
# Giả định mnist_model đã được tải lên
endpoint = aiplatform.Endpoint.create(display_name="mnist-endpoint")

endpoint.deploy(
    mnist_model,
    min_replica_count=1,
    max_replica_count=5,
    machine_type="n1-standard-4",
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=1
)
```

Đoạn mã này có thể mất vài phút để chạy, vì Vertex AI cần thiết lập một máy ảo. Trong ví dụ này, chúng ta sử dụng một máy khá cơ bản loại n1-standard-4 (xem <https://homl.info/machinetypes> để biết các loại khác). Chúng ta cũng sử dụng một GPU cơ bản loại NVIDIA_TESLA_K80 (xem <https://homl.info/accelerators> để biết các loại khác). Nếu bạn chọn một khu vực khác ngoài “us-central1”, thì bạn có thể cần thay đổi loại máy hoặc loại bộ tăng tốc thành các giá trị được hỗ trợ trong khu vực đó (ví dụ, không phải tất cả các khu vực đều có GPU Nvidia Tesla K80).

Vertex AI ban đầu sẽ tạo ra số lượng nút tính toán tối thiểu (chỉ một trong trường hợp này), và bất cứ khi nào số lượng truy vấn mỗi giây trở nên quá cao, nó sẽ tạo ra thêm các nút (lên đến

số lượng tối đa bạn đã định nghĩa, nằm trong trường hợp này) và sẽ cân bằng tải các truy vấn giữa chúng. Nếu tốc độ QPS giảm trong một thời gian, Vertex AI sẽ tự động dừng các nút tính toán bổ sung.

Chi phí do đó liên quan trực tiếp đến tải, cũng như loại máy và bộ tăng tốc bạn đã chọn và lượng dữ liệu bạn lưu trữ trên GCS. Mô hình định giá này rất phù hợp cho người dùng không thường xuyên và cho các dịch vụ có mức sử dụng tăng đột biến quan trọng. Nó cũng lý tưởng cho các công ty khởi nghiệp: giá vẫn thấp cho đến khi công ty khởi nghiệp thực sự bắt đầu hoạt động.

Chúc mừng, bạn đã triển khai mô hình đầu tiên của mình lên đám mây! Bây giờ hãy truy vấn dịch vụ dự đoán này:

```
# Giả định endpoint đã được deploy và X_new đã được chuẩn bị
response = endpoint.predict(instances=X_new.tolist())
```

Chúng ta đầu tiên cần chuyển đổi các ảnh chúng ta muốn phân loại thành một danh sách Python, như chúng ta đã làm trước đó khi gửi yêu cầu đến TF Serving bằng API REST. Đối tượng response chứa các dự đoán, được biểu diễn dưới dạng một danh sách các danh sách số float trong Python. Hãy làm tròn chúng đến hai chữ số thập phân và chuyển đổi chúng thành một mảng NumPy:

```
>>> import numpy as np

>>> np.round(response.predictions, 2)
array([[0. , 0. , 0. , 0. , 0. , 0. , 0. , 1. , 0. , 0. ],
       [0. , 0. , 0.99, 0.01, 0. , 0. , 0. , 0. , 0. , 0. ],
       [0. , 0.97, 0.01, 0. , 0. , 0. , 0. , 0.01, 0. , 0. ]])
```

Vâng! Chúng ta nhận được chính xác các dự đoán giống như trước đó. Bây giờ chúng ta có một dịch vụ dự đoán tuyệt vời đang chạy trên đám mây mà chúng ta có thể truy vấn từ bất cứ đâu một cách an toàn, và nó có thể tự động mở rộng lên hoặc xuống tùy thuộc vào số lượng QPS. Khi bạn hoàn thành việc sử dụng điểm cuối, đừng quên xóa nó, để tránh phải trả tiền vô ích:

```
endpoint.undeploy_all() # gỡ bỏ tất cả các mô hình khỏi điểm cuối
endpoint.delete()
```

Bây giờ hãy xem cách chạy một công việc trên Vertex AI để thực hiện dự đoán trên một lô dữ liệu có khả năng rất lớn.

Chạy các Công việc Dự đoán Hàng loạt trên Vertex AI

Nếu chúng ta có một số lượng lớn các dự đoán cần thực hiện, thay vì gọi dịch vụ dự đoán của chúng ta nhiều lần, chúng ta có thể yêu cầu Vertex AI chạy một công việc dự đoán hàng loạt cho chúng ta. Việc này không yêu cầu điểm cuối (endpoint), chỉ cần một mô hình. Ví dụ, hãy chạy một công việc dự đoán trên 100 ảnh đầu tiên của tập dữ liệu kiểm tra, sử dụng mô hình MNIST của chúng ta. Để làm điều này, trước tiên chúng ta cần chuẩn bị lô dữ liệu và tải nó

lên GCS. Một cách để làm điều này là tạo một tệp chứa một instance mỗi dòng, mỗi instance được định dạng dưới dạng giá trị JSON — định dạng này được gọi là JSON Lines — sau đó truyền tệp này cho Vertex AI. Vì vậy, hãy tạo một tệp JSON Lines trong một thư mục mới, sau đó tải thư mục này lên GCS:

```
from pathlib import Path
import json
# Giả định X_test đã được định nghĩa, ví dụ: X_test = tf.keras.datasets.mnist
# .load_data()[1][0]
# Giả định bucket đã được tạo và hàm upload_directory đã được định nghĩa

batch_path = Path("my_mnist_batch")
batch_path.mkdir(exist_ok=True)

with open(batch_path / "my_mnist_batch.jsonl", "w") as jsonl_file:
    for image in X_test[:100].tolist():
        jsonl_file.write(json.dumps(image))
        jsonl_file.write("\n")

upload_directory(bucket, batch_path)
```

Bây giờ chúng ta đã sẵn sàng khởi chạy công việc dự đoán, chỉ định tên công việc, loại và số lượng máy và bộ tăng tốc cần sử dụng, đường dẫn GCS đến tệp JSON Lines mà chúng ta vừa tạo, và đường dẫn đến thư mục GCS nơi Vertex AI sẽ lưu các dự đoán của mô hình:

```
# Giả định mnist_model đã được tải lên Vertex AI trước đó
# Giả định bucket_name và batch_path đã được định nghĩa

batch_prediction_job = mnist_model.batch_predict(
    job_display_name="my_batch_prediction_job",
    machine_type="n1-standard-4",
    starting_replica_count=1,
    max_replica_count=5,
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=1,
    gcs_source=[f"gs://{bucket_name}/{batch_path.name}/my_mnist_batch.jsonl"]
,
    gcs_destination_prefix=f"gs://{bucket_name}/my_mnist_predictions/",
    sync=True # đặt thành False nếu bạn không muốn chờ hoàn thành
)
```

Việc này sẽ mất vài phút, chủ yếu là để tạo các nút tính toán trên Vertex AI. Khi lệnh này hoàn tất, các dự đoán sẽ có sẵn trong một tập hợp các tệp có tên giống như `prediction.results-00001-of-00002`.

Các tệp này sử dụng định dạng JSON Lines theo mặc định, và mỗi giá trị là một dictionary chứa một instance và dự đoán tương ứng của nó (tức là 10 xác suất). Các instance được liệt kê theo cùng thứ tự với các đầu vào. Công việc cũng xuất ra các tệp `prediction-errors*`, có thể hữu ích cho việc gỡ lỗi nếu có gì đó không ổn. Chúng ta có thể lặp qua tất cả các tệp

đầu ra này bằng cách sử dụng `batch_prediction_job.iter_outputs()`, vì vậy hãy đi qua tất cả các dự đoán và lưu trữ chúng trong một mảng `y_probas`:

```
import numpy as np
import json
# Giả định batch_prediction_job đã hoàn thành

y_probas = []

for blob in batch_prediction_job.iter_outputs():
    if "prediction.results" in blob.name:
        for line in blob.download_as_text().splitlines():
            y_proba = json.loads(line)["prediction"]
            y_probas.append(y_proba)
```

Bây giờ hãy xem các dự đoán này tốt đến mức nào:

```
# Giả định y_test đã được định nghĩa từ trước, ví dụ: y_test = tf.keras.datas
ets.mnist.load_data()[1][1]
>>> y_pred = np.argmax(y_probas, axis=1)

>>> accuracy = np.sum(y_pred == y_test[:100]) / 100
0.98
```

Tuyệt vời, độ chính xác 98%!

Định dạng JSON Lines là mặc định, nhưng khi xử lý các instance lớn như ảnh, nó quá dài dòng. May mắn thay, phương thức `batch_predict()` chấp nhận một đối số `instances_format` cho phép bạn chọn một định dạng khác nếu muốn. Nó mặc định là “jsonl”, nhưng bạn có thể thay đổi nó thành “csv”, “tf-record”, “tf-record-gzip”, “bigquery”, hoặc “file-list”.

Nếu bạn đặt nó thành “file-list”, thì đối số `gcs_source` sẽ trỏ đến một tệp văn bản chứa một đường dẫn tệp đầu vào mỗi dòng; ví dụ, trỏ đến các tệp ảnh PNG. Vertex AI sẽ đọc các tệp này dưới dạng nhị phân, mã hóa chúng bằng Base64, và truyền các chuỗi byte kết quả cho mô hình. Điều này có nghĩa là bạn phải thêm một lớp tiền xử lý trong mô hình của mình để phân tích cú pháp các chuỗi Base64, bằng cách sử dụng `tf.io.decode_base64()`. Nếu các tệp là ảnh, bạn phải phân tích cú pháp kết quả bằng một hàm như `tf.io.decode_image()` hoặc `tf.io.decode_png()`, như đã thảo luận trong Chương 13.

Khi bạn đã hoàn thành việc sử dụng mô hình, bạn có thể xóa nó nếu muốn, bằng cách chạy `mnist_model.delete()`. Bạn cũng có thể xóa các thư mục bạn đã tạo trong GCS bucket của mình, tùy chọn xóa chính bucket (nếu nó trống), và công việc dự đoán hàng loạt:

```
# Giả định bucket và batch_prediction_job đã được định nghĩa

for prefix in ["my_mnist_model/", "my_mnist_batch/", "my_mnist_predictions/"]
:
    blobs = bucket.list_blobs(prefix=prefix)
```

```
for blob in blobs:
    blob.delete()

# Chỉ xóa bucket nếu nó trống
# bucket.delete()

batch_prediction_job.delete()
```

Bây giờ bạn đã biết cách triển khai một mô hình lên Vertex AI, tạo dịch vụ dự đoán và chạy các công việc dự đoán hàng loạt. Nhưng nếu bạn muốn triển khai mô hình của mình vào một ứng dụng di động thì sao? Hoặc vào một thiết bị nhúng, chẳng hạn như hệ thống điều khiển nhiệt độ, thiết bị theo dõi thể dục, hay một chiếc xe tự lái?

Triển khai Mô hình lên Thiết bị Di động hoặc Thiết bị Nhúng

Các mô hình học máy không chỉ giới hạn ở việc chạy trên các máy chủ tập trung lớn với nhiều GPU: chúng có thể chạy gần nguồn dữ liệu hơn (điều này được gọi là điện toán biên - edge computing), ví dụ trong thiết bị di động của người dùng hoặc trong một thiết bị nhúng. Có nhiều lợi ích khi phân tán các phép tính và di chuyển chúng đến biên: nó cho phép thiết bị trở nên thông minh ngay cả khi không được kết nối internet, nó giảm độ trễ bằng cách không phải gửi dữ liệu đến máy chủ từ xa và giảm tải trên các máy chủ, và nó có thể cải thiện quyền riêng tư, vì dữ liệu của người dùng có thể ở lại trên thiết bị.

Tuy nhiên, việc triển khai mô hình đến biên cũng có những nhược điểm. Tài nguyên tính toán của thiết bị thường rất nhỏ so với một máy chủ nhiều GPU mạnh mẽ. Một mô hình lớn có thể không vừa với thiết bị, nó có thể sử dụng quá nhiều RAM và CPU, và có thể mất quá nhiều thời gian để tải xuống. Kết quả là, ứng dụng có thể trở nên không phản hồi, và thiết bị có thể nóng lên và nhanh chóng hết pin. Để tránh tất cả những điều này, bạn cần tạo một mô hình nhẹ và hiệu quả, mà không phải hy sinh quá nhiều độ chính xác. Thư viện TFLite cung cấp một số công cụ để giúp bạn triển khai mô hình của mình đến biên, với ba mục tiêu chính:

- **Giảm kích thước mô hình**, để rút ngắn thời gian tải xuống và giảm sử dụng RAM.
- **Giảm lượng tính toán cần thiết cho mỗi dự đoán**, để giảm độ trễ, sử dụng pin và nhiệt độ.
- **Thích nghi mô hình với các ràng buộc cụ thể của thiết bị**.

Để giảm kích thước mô hình, bộ chuyển đổi mô hình của TFLite có thể lấy một SavedModel và nén nó thành một định dạng nhẹ hơn nhiều dựa trên FlatBuffers. Đây là một thư viện tuần tự hóa đa nền tảng hiệu quả (hơi giống protocol buffers) ban đầu được Google tạo ra cho trò chơi. Nó được thiết kế để bạn có thể tải FlatBuffers trực tiếp vào RAM mà không cần bất kỳ tiền xử lý nào: điều này giúp giảm thời gian tải và dấu vết bộ nhớ.

Khi mô hình được tải vào một thiết bị di động hoặc thiết bị nhúng, trình thông dịch TFLite sẽ thực thi nó để đưa ra các dự đoán. Đây là cách bạn có thể chuyển đổi một SavedModel thành FlatBuffer và lưu nó vào tệp `.tflite`:

```
import tensorflow as tf
# Giả định model_path đã được định nghĩa từ bước xuất SavedModel

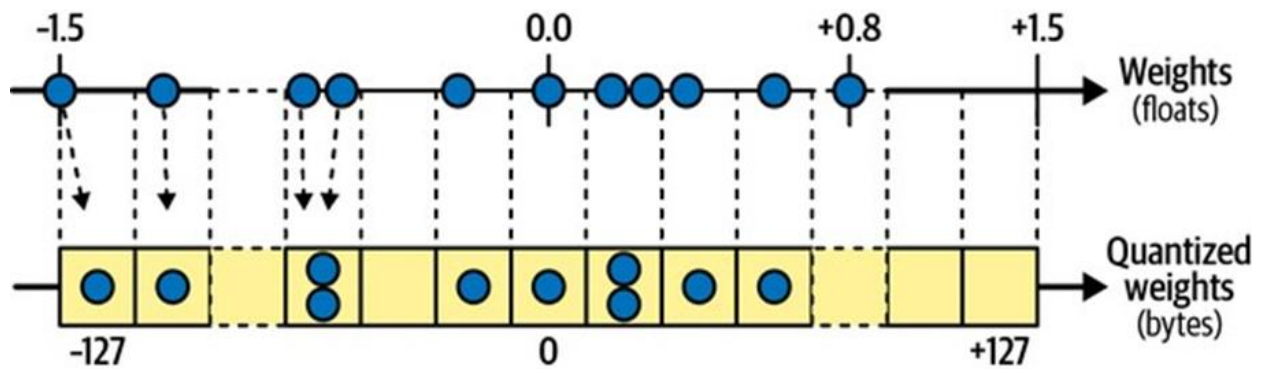
converter = tf.lite.TFLiteConverter.from_saved_model(str(model_path))
tflite_model = converter.convert()

with open("my_converted_savedmodel.tflite", "wb") as f:
    f.write(tflite_model)
```

Bộ chuyển đổi cũng tối ưu hóa mô hình, cả để thu nhỏ nó và để giảm độ trễ. Nó loại bỏ tất cả các hoạt động không cần thiết để đưa ra dự đoán (chẳng hạn như các hoạt động huấn luyện), và nó tối ưu hóa các phép tính bất cứ khi nào có thể; ví dụ, $3timesa + 4timesa + 5timesa$ sẽ được chuyển đổi thành $12timesa$. Ngoài ra, nó cố gắng hợp nhất các hoạt động bất cứ khi nào có thể. Ví dụ, nếu có thể, các lớp chuẩn hóa hàng loạt (batch normalization layers) cuối cùng được gộp vào các phép cộng và nhân của lớp trước đó. Để có ý tưởng hay về mức độ TFLite có thể tối ưu hóa một mô hình, hãy tải xuống một trong các mô hình TFLite đã được huấn luyện trước, chẳng hạn như Inception_V1_quant (nhấp vào [tflite&pb](#)), giải nén tệp lưu trữ, sau đó mở công cụ trực quan hóa đồ thị Netron tuyệt vời và tải tệp .pb lên để xem mô hình gốc. Đó là một đồ thị lớn, phức tạp, phải không? Tiếp theo, mở mô hình `optimized.tflite` và chiêm ngưỡng vẻ đẹp của nó!

Một cách khác bạn có thể giảm kích thước mô hình — ngoài việc đơn giản là sử dụng các kiến trúc mạng nơ-ron nhỏ hơn — là bằng cách sử dụng độ rộng bit nhỏ hơn: ví dụ, nếu bạn sử dụng half-floats (16 bit) thay vì regular floats (32 bit), kích thước mô hình sẽ giảm đi 2 lần, với chi phí là sự sụt giảm độ chính xác (thường là nhỏ). Hơn nữa, quá trình huấn luyện sẽ nhanh hơn, và bạn sẽ sử dụng khoảng một nửa lượng RAM GPU.

Bộ chuyển đổi của TFLite có thể đi xa hơn nữa, bằng cách lượng tử hóa các trọng số mô hình xuống các số nguyên 8-bit, dấu phẩy thập! Điều này dẫn đến giảm kích thước gấp 4 lần so với việc sử dụng số float 32-bit. Cách tiếp cận đơn giản nhất được gọi là lượng tử hóa sau huấn luyện (post-training quantization): nó chỉ lượng tử hóa các trọng số sau khi huấn luyện, sử dụng một kỹ thuật lượng tử hóa đối xứng khá cơ bản nhưng hiệu quả. Nó tìm giá trị tuyệt đối tối đa của trọng số, m , sau đó nó ánh xạ dải số float từ $-m$ đến $+m$ sang dải số nguyên (fixed-point) từ -127 đến $+127$. Ví dụ, nếu các trọng số nằm trong khoảng từ -1.5 đến $+0.8$, thì các byte $-127, 0, textva` + 127$  sẽ tương ứng với các số float  $-1.5, 0.0, textva` + 1.5$, tương ứng (xem Hình 19-5). Lưu ý rằng 0.0 luôn ánh xạ đến 0 khi sử dụng lượng tử hóa đối xứng. Cũng lưu ý rằng các giá trị byte từ $+68$ đến $+127$ sẽ không được sử dụng trong ví dụ này, vì chúng ánh xạ đến các số float lớn hơn $+0.8$.



Hình 19-5. Từ số float 32-bit sang số nguyên 8-bit, sử dụng lượng tử hóa đối xứng.

Để thực hiện lượng tử hóa sau huấn luyện này, bạn chỉ cần thêm `DEFAULT` vào danh sách các tối ưu hóa của bộ chuyển đổi trước khi gọi phương thức `convert()`:

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
```

Kỹ thuật này giảm đáng kể kích thước mô hình, giúp tải xuống nhanh hơn nhiều và sử dụng ít không gian lưu trữ hơn. Khi chạy, các trọng số đã được lượng tử hóa sẽ được chuyển đổi trở lại thành số float trước khi chúng được sử dụng. Các số float được khôi phục này không hoàn toàn giống với các số float gốc, nhưng chúng không quá khác biệt, vì vậy việc mất độ chính xác thường có thể chấp nhận được. Để tránh tính toán lại các giá trị float mọi lúc, điều này sẽ làm chậm mô hình nghiêm trọng, TFLite lưu trữ chúng: thật không may, điều này có nghĩa là kỹ thuật này không làm giảm việc sử dụng RAM, và nó cũng không làm tăng tốc mô hình. Nó chủ yếu hữu ích để giảm kích thước ứng dụng.

Cách hiệu quả nhất để giảm độ trễ và tiêu thụ điện năng là cũng lượng tử hóa các kích hoạt (activations) để các phép tính có thể được thực hiện hoàn toàn bằng số nguyên, mà không cần bất kỳ phép toán dấu phẩy động nào. Ngay cả khi sử dụng cùng độ rộng bit (ví dụ: số nguyên 32-bit thay vì số float 32-bit), các phép tính số nguyên sử dụng ít chu kỳ CPU hơn, tiêu thụ ít năng lượng hơn và tạo ra ít nhiệt hơn. Và nếu bạn cũng giảm độ rộng bit (ví dụ: xuống số nguyên 8-bit), bạn có thể nhận được tốc độ tăng đáng kể. Hơn nữa, một số thiết bị tăng tốc mạng nơ-ron — chẳng hạn như Edge TPU của Google — chỉ có thể xử lý số nguyên, vì vậy việc lượng tử hóa hoàn toàn cả trọng số và kích hoạt là bắt buộc. Điều này có thể được thực hiện sau huấn luyện; nó yêu cầu một bước hiệu chuẩn để tìm giá trị tuyệt đối tối đa của các kích hoạt, vì vậy bạn cần cung cấp một mẫu dữ liệu huấn luyện đại diện cho TFLite (nó không cần phải quá lớn), và nó sẽ xử lý dữ liệu thông qua mô hình và đo lường các thống kê kích hoạt cần thiết cho việc lượng tử hóa. Bước này thường nhanh.

Vấn đề chính với lượng tử hóa là nó làm mất một chút độ chính xác: nó tương tự như việc thêm nhiễu vào trọng số và kích hoạt. Nếu việc mất độ chính xác quá nghiêm trọng, thì bạn có thể cần sử dụng **lượng tử hóa có nhận thức huấn luyện (quantization-aware training)**. Điều này có nghĩa là thêm các phép toán lượng tử hóa giả vào mô hình để nó có thể học cách bỏ qua nhiễu lượng tử hóa trong quá trình huấn luyện; các trọng số cuối cùng sau đó sẽ mạnh mẽ hơn đối với lượng tử hóa. Hơn nữa, bước hiệu chuẩn có thể được tự động hóa trong quá trình huấn luyện, giúp đơn giản hóa toàn bộ quá trình.

Tôi đã giải thích các khái niệm cốt lõi của TFLite, nhưng việc đi sâu vào mã hóa một ứng dụng di động hoặc thiết bị nhúng sẽ đòi hỏi một cuốn sách chuyên biệt. May mắn thay, một số cuốn đã tồn tại: nếu bạn muốn tìm hiểu thêm về xây dựng ứng dụng TensorFlow cho thiết bị di động và thiết bị nhúng, hãy xem các cuốn sách của O'Reilly là *TinyML: Machine Learning with TensorFlow on Arduino and Ultra-Low Power Micro-Controllers* của Pete Warden (cựu trưởng nhóm TFLite) và Daniel Situnayake, và *AI and Machine Learning for On-Device Development* của Laurence Moroney.

Bây giờ nếu bạn muốn sử dụng mô hình của mình trong một trang web, chạy trực tiếp trong trình duyệt của người dùng thì sao?

Công thức Toán (Phán đoán):

7. **Tính độ chính xác (Accuracy):** Độ chính xác được tính bằng tỷ lệ số dự đoán đúng trên tổng số dự đoán.
$$\text{accuracy} = \frac{\sum (y_{\text{pred}} == y_{\text{test}}[i])}{N}$$
 Trong đoạn mã: `accuracy = np.sum(y_pred == y_test[:100]) / 100` Ở đây, $N = 100$ (100 ảnh đầu tiên), $y_{\text{pred}, i}$ là dự đoán của mô hình cho ảnh thứ i , và $y_{\text{true}, i}$ là nhãn thực của ảnh thứ i . Hàm `mathbb{I}(\cdot)` là hàm chỉ báo, trả về 1 nếu điều kiện đúng, và 0 nếu sai.
8. **Tối ưu hóa phép tính trong TFLite:** Khi TFLite tối ưu hóa các phép tính, ví dụ: $3 \times a + 4 \times a + 5 \times a$ được chuyển đổi thành $12 \times a$. Đây là một dạng rút gọn đại số cơ bản: $3a + 4a + 5a = (3+4+5)a = 12a$. Đây là một tối ưu hóa đơn giản nhưng hiệu quả, giảm số lượng phép nhân và cộng cần thiết.
9. **Lượng tử hóa đối xứng (Symmetrical Quantization):** Quá trình ánh xạ giá trị dấu phẩy động (x_{float}) sang số nguyên 8-bit (x_{int8}) trong khoảng từ -127 đến $+127$, dựa trên giá trị tuyệt đối tối đa m . Giả sử dải giá trị float là $[-m, +m]$. Để ánh xạ nó sang $[-127, +127]$, chúng ta có thể sử dụng công thức:
$$x_{\text{int8}} = \lfloor \frac{x_{\text{float}} \times 127}{m} \rfloor$$
 Và khi chuyển đổi ngược lại từ x_{int8} sang x_{float} (để sử dụng tại runtime):
$$x_{\text{float}} = \frac{x_{\text{int8}} \times m}{127}$$
 Trong ví dụ: nếu trọng số từ -1.5 đến $+0.8$, thì m sẽ là 1.5 (giá trị tuyệt đối lớn nhất).
 - $x_{\text{float}} = -1.5 \implies x_{\text{int8}} = \text{round}(\frac{-1.5 \times 127}{1.5}) = \text{round}(-127) = -127$
 - $x_{\text{float}} = 0.0 \implies x_{\text{int8}} = \text{round}(\frac{0.0 \times 127}{1.5}) = \text{round}(0) = 0$
 - $x_{\text{float}} = +1.5 \implies x_{\text{int8}} = \text{round}(\frac{1.5 \times 127}{1.5}) = \text{round}(127) = +127$ (Lưu ý: ví dụ gốc có $+0.8$, nhưng để ánh xạ tới $+127$ thì phải là $+1.5$ nếu $m=1.5$) Mục tiêu của lượng tử hóa là giảm kích thước mô hình (ví dụ: từ 32 bit float xuống 8 bit integer, giảm 4 lần kích thước) và tăng tốc độ tính toán (phép tính số nguyên nhanh hơn phép tính số thực), cũng như giảm tiêu thụ năng lượng.

Chạy Mô hình trong Trang Web

Chạy mô hình học máy của bạn ở phía máy khách, trong trình duyệt của người dùng, thay vì ở phía máy chủ có thể hữu ích trong nhiều tình huống, chẳng hạn như:

- Khi ứng dụng web của bạn thường được sử dụng trong các tình huống mà kết nối của người dùng bị gián đoạn hoặc chậm (ví dụ: một trang web dành cho người đi bộ đường dài), vì vậy chạy mô hình trực tiếp ở phía máy khách là cách duy nhất để làm cho trang web của bạn đáng tin cậy.
- Khi bạn cần phản hồi của mô hình nhanh nhất có thể (ví dụ: đối với một trò chơi trực tuyến). Việc loại bỏ nhu cầu truy vấn máy chủ để đưa ra dự đoán chắc chắn sẽ giảm độ trễ và làm cho trang web phản hồi nhanh hơn nhiều.
- Khi dịch vụ web của bạn đưa ra dự đoán dựa trên một số dữ liệu riêng tư của người dùng, và bạn muốn bảo vệ quyền riêng tư của người dùng bằng cách đưa ra dự đoán ở phía máy khách để dữ liệu riêng tư không bao giờ phải rời khỏi máy của người dùng.

Đối với tất cả các kịch bản này, bạn có thể sử dụng thư viện TensorFlow.js (TFJS) JavaScript. Thư viện này có thể tải một mô hình TFLite và đưa ra dự đoán trực tiếp trong trình duyệt của người dùng. Ví dụ, module JavaScript sau đây import thư viện TFJS, tải xuống một mô hình MobileNet đã được huấn luyện trước, và sử dụng mô hình này để phân loại một ảnh và ghi lại các dự đoán. Bạn có thể thử nghiệm với mã tại <https://homl.info/tfjscode>, sử dụng Glitch.com, một trang web cho phép bạn xây dựng ứng dụng web trong trình duyệt của mình miễn phí; nhấp vào nút PREVIEW ở góc dưới bên phải của trang để xem mã hoạt động:

```
import * as tf from "https://cdn.jsdelivr.net/npm/@tensorflow/tfjs@latest";
import * as mobilenet from "https://cdn.jsdelivr.net/npm/@tensorflow-models/mobilenet@1.0.0";

const image = document.getElementById("image");

mobilenet.load().then(model => {
  model.classify(image).then(predictions => {
    for (var i = 0; i < predictions.length; i++) {
      let className = predictions[i].className;
      let proba = (predictions[i].probability * 100).toFixed(1);
      console.log(className + " : " + proba + "%");
    }
  });
});
```

Thậm chí có thể biến trang web này thành một ứng dụng web tiến bộ (PWA): đây là một trang web tuân thủ một số tiêu chí cho phép nó được xem trong bất kỳ trình duyệt nào, và thậm chí được cài đặt như một ứng dụng độc lập trên thiết bị di động. Ví dụ, hãy thử truy cập <https://homl.info/tfjswpa> trên một thiết bị di động: hầu hết các trình duyệt hiện đại sẽ hỏi bạn có muốn thêm “TFJS Demo” vào màn hình chính của bạn hay không. Nếu bạn chấp nhận, bạn sẽ thấy một biểu tượng mới trong danh sách ứng dụng của mình. Nhấp vào biểu tượng này sẽ tải trang web “TFJS Demo” bên trong cửa sổ riêng của nó, giống như một ứng

dụng di động thông thường. Một PWA thậm chí có thể được cấu hình để hoạt động ngoại tuyến, bằng cách sử dụng một service worker: đây là một module JavaScript chạy trong luồng riêng biệt của nó trong trình duyệt và chặn các yêu cầu mạng, cho phép nó lưu trữ tài nguyên để PWA có thể chạy nhanh hơn, hoặc thậm chí hoàn toàn ngoại tuyến. Nó cũng có thể gửi tin nhắn đẩy, chạy các tác vụ trong nền, và nhiều hơn nữa. PWAs cho phép bạn quản lý một cơ sở mã duy nhất cho web và cho thiết bị di động. Chúng cũng giúp dễ dàng đảm bảo rằng tất cả người dùng chạy cùng một phiên bản của ứng dụng của bạn. Bạn có thể thử nghiệm với mã PWA của TFJS Demo này trên Glitch.com tại <https://homl.info/wpacode>.

TFJS cũng hỗ trợ huấn luyện mô hình trực tiếp trong trình duyệt web của bạn! Và nó thực sự khá nhanh. Nếu máy tính của bạn có card GPU, thì TFJS thường có thể sử dụng nó, ngay cả khi đó không phải là card Nvidia. Thật vậy, TFJS sẽ sử dụng WebGL khi nó khả dụng, và vì các trình duyệt web hiện đại thường hỗ trợ một loạt các card GPU, TFJS thực sự hỗ trợ nhiều loại card GPU hơn TensorFlow thông thường (chỉ hỗ trợ card Nvidia).

Huấn luyện một mô hình trong trình duyệt web của người dùng có thể đặc biệt hữu ích để đảm bảo rằng dữ liệu của người dùng này vẫn riêng tư. Một mô hình có thể được huấn luyện tập trung, và sau đó được tinh chỉnh cục bộ, trong trình duyệt, dựa trên dữ liệu của người dùng đó. Nếu bạn quan tâm đến chủ đề này, hãy tìm hiểu về học liên kết (federated learning). Một lần nữa, để làm rõ chủ đề này sẽ cần cả một cuốn sách. Nếu bạn muốn tìm hiểu thêm về TensorFlow.js, hãy xem các cuốn sách của O'Reilly là *Practical Deep Learning for Cloud, Mobile, and Edge*, của Anirudh Koul et al., hoặc *Learning TensorFlow.js*, của Gant Laborde.

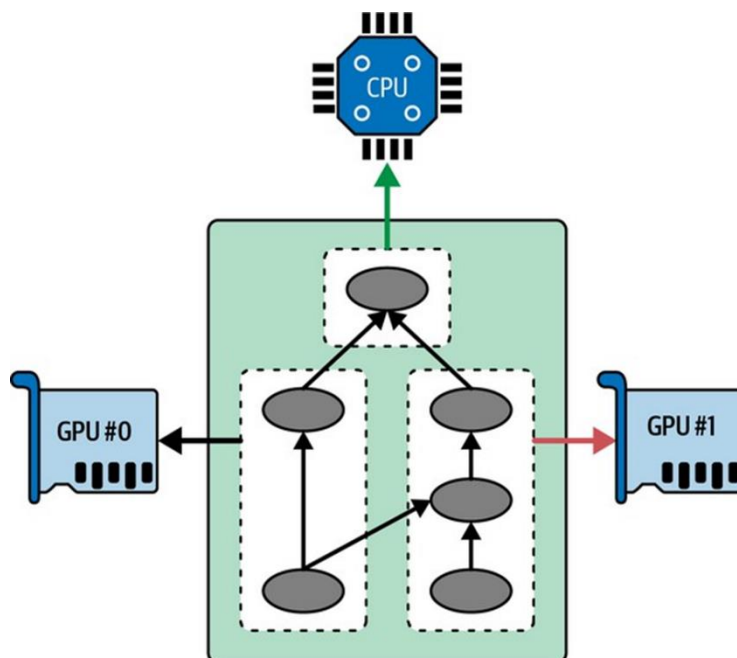
Bây giờ bạn đã thấy cách triển khai các mô hình TensorFlow lên TF Serving, hoặc lên đám mây với Vertex AI, hoặc lên các thiết bị di động và nhúng bằng TFLite, hoặc lên trình duyệt web bằng TFJS, hãy thảo luận về cách sử dụng GPU để tăng tốc tính toán.

Sử dụng GPU để Tăng tốc Tính toán

Trong Chương 11, chúng ta đã xem xét một số kỹ thuật có thể tăng tốc đáng kể quá trình huấn luyện: khởi tạo trọng số tốt hơn, các bộ tối ưu hóa tinh vi, v.v. Nhưng ngay cả với tất cả các kỹ thuật này, việc huấn luyện một mạng nơ-ron lớn trên một máy với một CPU duy nhất có thể mất hàng giờ, hàng ngày, hoặc thậm chí hàng tuần, tùy thuộc vào tác vụ. Nhờ GPU, thời gian huấn luyện này có thể giảm xuống còn vài phút hoặc vài giờ. Điều này không chỉ tiết kiệm một lượng thời gian khổng lồ, mà còn có nghĩa là bạn có thể thử nghiệm với các mô hình khác nhau dễ dàng hơn nhiều, và thường xuyên huấn luyện lại mô hình của bạn trên dữ liệu mới.

Trong các chương trước, chúng ta đã sử dụng các môi trường chạy có hỗ trợ GPU trên Google Colab. Tất cả những gì bạn phải làm để có được điều này là chọn “Change runtime type” từ menu Runtime, và chọn loại bộ tăng tốc GPU; TensorFlow tự động phát hiện GPU và sử dụng nó để tăng tốc tính toán, và mã hoàn toàn giống như khi không có GPU. Sau đó, trong chương này, bạn đã thấy cách triển khai các mô hình của mình lên Vertex AI trên nhiều nút tính toán có hỗ trợ GPU: đó chỉ là vấn đề chọn đúng ảnh Docker có hỗ trợ GPU khi tạo mô hình Vertex AI, và chọn loại GPU mong muốn khi gọi `endpoint.deploy()`. Nhưng nếu bạn muốn mua GPU của riêng mình thì sao? Và nếu bạn muốn phân tán các phép tính trên CPU và nhiều thiết bị GPU trên một máy duy nhất (xem Hình 19-6)? Đây là điều chúng ta sẽ thảo luận bây

giờ, sau đó trong chương này chúng ta sẽ thảo luận về cách phân tán các phép tính trên nhiều máy chủ.



Hình 19-6. Thực thi đồ thị TensorFlow trên nhiều thiết bị song song.

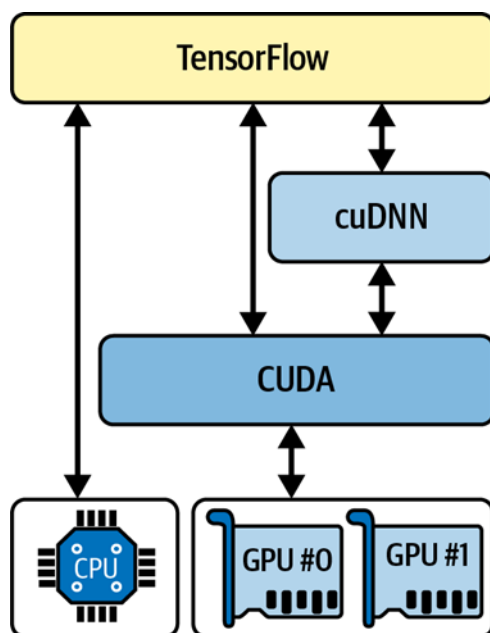
Có được GPU của riêng bạn

Nếu bạn biết rằng mình sẽ sử dụng GPU nhiều và trong thời gian dài, thì việc mua GPU của riêng bạn có thể có ý nghĩa tài chính. Bạn cũng có thể muốn huấn luyện các mô hình của mình cục bộ vì bạn không muốn tải dữ liệu của mình lên đám mây. Hoặc có lẽ bạn chỉ muốn mua một card GPU để chơi game, và bạn cũng muốn sử dụng nó cho học sâu.

Nếu bạn quyết định mua một card GPU, thì hãy dành thời gian để đưa ra lựa chọn đúng đắn. Bạn sẽ cần xem xét lượng RAM bạn sẽ cần cho các tác vụ của mình (ví dụ: thường ít nhất 10 GB cho xử lý ảnh hoặc NLP), băng thông (tức là bạn có thể gửi dữ liệu vào và ra GPU nhanh như thế nào), số lượng lõi, hệ thống làm mát, v.v. Tim Dettmers đã viết một bài đăng blog tuyệt vời để giúp bạn lựa chọn: tôi khuyến khích bạn đọc kỹ nó. Tại thời điểm viết bài này, TensorFlow chỉ hỗ trợ các card Nvidia với CUDA Compute Capability 3.5+ (cũng như TPUs của Google, tất nhiên), nhưng nó có thể mở rộng hỗ trợ cho các nhà sản xuất khác, vì vậy hãy đảm bảo kiểm tra tài liệu của TensorFlow để xem thiết bị nào được hỗ trợ hiện nay.

Nếu bạn chọn một card GPU Nvidia, bạn sẽ cần cài đặt các driver Nvidia và một số thư viện Nvidia phù hợp. Những thứ này bao gồm thư viện CUDA Toolkit (Compute Unified Device Architecture), cho phép các nhà phát triển sử dụng GPU có hỗ trợ CUDA cho tất cả các loại tính toán (không chỉ tăng tốc đồ họa), và thư viện CUDA Deep Neural Network (cuDNN), một thư viện tăng tốc GPU cho các phép tính DNN phổ biến như các lớp kích hoạt, chuẩn hóa, tích chập xuôi và ngược, và gộp (pooling) (xem Chương 14). cuDNN là một phần của Nvidia's Deep Learning SDK. Lưu ý rằng bạn sẽ cần tạo một tài khoản nhà phát triển Nvidia để tải

xuống nó. TensorFlow sử dụng CUDA và cuDNN để điều khiển các card GPU và tăng tốc tính toán (xem Hình 19-7).



Hình 19-7. TensorFlow sử dụng CUDA và cuDNN để điều khiển GPU và tăng cường DNN.

Khi bạn đã cài đặt (các) card GPU và tất cả các driver và thư viện cần thiết, bạn có thể sử dụng lệnh `nvidia-smi` để kiểm tra xem mọi thứ đã được cài đặt đúng cách chưa. Lệnh này liệt kê các card GPU khả dụng, cũng như tất cả các tiến trình đang chạy trên mỗi card. Trong ví dụ này, đó là một card GPU Nvidia Tesla T4 với khoảng 15 GB RAM khả dụng, và hiện không có tiến trình nào đang chạy trên đó:

```

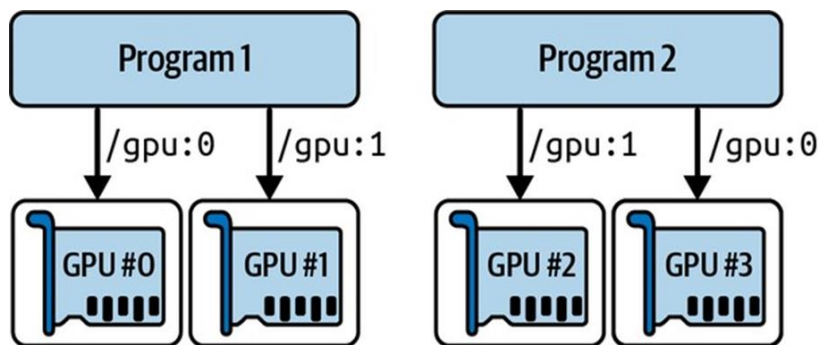
$ nvidia-smi
Sun Apr 10 04:52:10 2022
+-----+
+---+
| NVIDIA-SMI 460.32.03    Driver Version: 460.32.03          CUDA Version: 11.2     |
|-----+-----+-----+
+---+
| GPU  Name            Persistence-M| Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf   Pwr:Usage/Cap|      Memory-Usage | GPU-Util  Compute M. |
|                               |                  |                MIG M. |
|=====+=====+=====+
==|
|   0   Tesla T4               Off  | 00000000:00:04.0 Off  |
0 |
| N/A   34C    P8             9W / 70W |  3MiB / 15109MiB |      0%    Defaul

```



```
$ CUDA_DEVICE_ORDER=PCI_BUS_ID CUDA_VISIBLE_DEVICES=0,1 python3 program_1.py
# và trong một terminal khác:
$ CUDA_DEVICE_ORDER=PCI_BUS_ID CUDA_VISIBLE_DEVICES=3,2 python3 program_2.py
```

Chương trình 1 sau đó sẽ chỉ thấy các card GPU 0 và 1, được đặt tên lần lượt là “/gpu:0” và “/gpu:1” trong TensorFlow, và chương trình 2 sẽ chỉ thấy các card GPU 2 và 3, được đặt tên lần lượt là “/gpu:1” và “/gpu:0” (lưu ý thứ tự). Mọi thứ sẽ hoạt động tốt (xem Hình 19-8). Tất nhiên, bạn cũng có thể định nghĩa các biến môi trường này trong Python bằng cách đặt `os.environ["CUDA_DEVICE_ORDER"]` và `os.environ["CUDA_VISIBLE_DEVICES"]`, miễn là bạn thực hiện trước khi sử dụng TensorFlow.



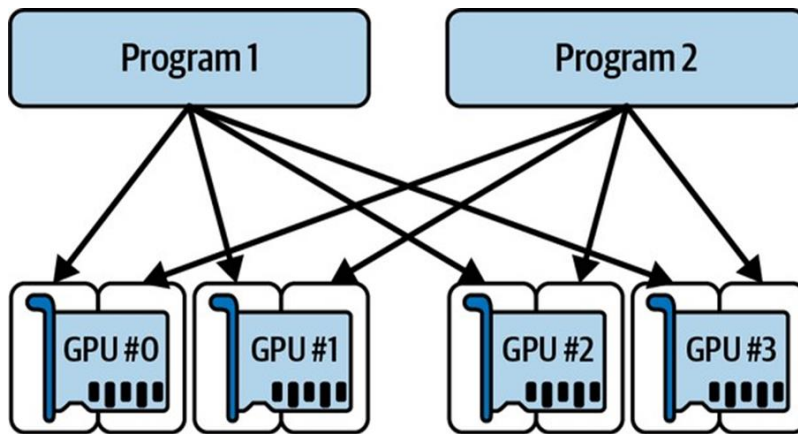
Hình 19-8. Mỗi chương trình nhận hai GPU.

Một tùy chọn khác là yêu cầu TensorFlow chỉ chiếm một lượng RAM GPU cụ thể. Điều này phải được thực hiện ngay sau khi import TensorFlow. Ví dụ, để yêu cầu TensorFlow chỉ chiếm 2 GiB RAM trên mỗi GPU, bạn phải tạo một thiết bị GPU logic (đôi khi được gọi là thiết bị GPU ảo) cho mỗi thiết bị GPU vật lý và đặt giới hạn bộ nhớ của nó là 2 GiB (tức là 2.048 MiB):

```
import tensorflow as tf
# Giả định physical_gpus đã được lấy bằng tf.config.list_physical_devices("GPU")

for gpu in physical_gpus:
    tf.config.set_logical_device_configuration(
        gpu, [tf.config.LogicalDeviceConfiguration(memory_limit=2048)]
    )
```

Giả sử bạn có bốn GPU, mỗi GPU có ít nhất 4 GiB RAM: trong trường hợp này, hai chương trình như thế này có thể chạy song song, mỗi chương trình sử dụng cả bốn card GPU (xem Hình 19-9). Nếu bạn chạy lệnh `nvidia-smi` trong khi cả hai chương trình đang chạy, bạn sẽ thấy rằng mỗi tiến trình giữ 2 GiB RAM trên mỗi card.



Hình 19-9. Mỗi chương trình nhận tất cả bốn GPU, nhưng chỉ với 2 GiB RAM trên mỗi GPU.

Một tùy chọn khác nữa là yêu cầu TensorFlow chỉ chiếm bộ nhớ khi nó cần. Một lần nữa, điều này phải được thực hiện ngay sau khi import TensorFlow:

```
import tensorflow as tf
# Giả định physical_gpus đã được lấy

for gpu in physical_gpus:
    tf.config.experimental.set_memory_growth(gpu, True)
```

Một cách khác để làm điều này là đặt biến môi trường `TF_FORCE_GPU_ALLOW_GROWTH` thành `true`. Với tùy chọn này, TensorFlow sẽ không bao giờ giải phóng bộ nhớ một khi nó đã chiếm (một lần nữa, để tránh phân mảnh bộ nhớ), ngoại trừ tất nhiên khi chương trình kết thúc. Có thể khó hơn để đảm bảo hành vi xác định khi sử dụng tùy chọn này (ví dụ: một chương trình có thể gặp sự cố vì mức sử dụng bộ nhớ của chương trình khác tăng vọt), vì vậy trong sản xuất, bạn có thể sẽ muốn gắn bó với một trong các tùy chọn trước đó. Tuy nhiên, có một số trường hợp nó rất hữu ích: ví dụ, khi bạn sử dụng một máy để chạy nhiều sổ tay Jupyter, một số trong số đó sử dụng TensorFlow. Biến môi trường `TF_FORCE_GPU_ALLOW_GROWTH` được đặt thành `true` trong các môi trường chạy Colab.

Cuối cùng, trong một số trường hợp, bạn có thể muốn chia một GPU thành hai hoặc nhiều thiết bị logic. Ví dụ, điều này hữu ích nếu bạn chỉ có một GPU vật lý — như trong môi trường chạy Colab — nhưng bạn muốn kiểm tra một thuật toán đa GPU. Đoạn mã sau chia GPU #0 thành hai thiết bị logic, mỗi thiết bị có 2 GiB RAM (một lần nữa, điều này phải được thực hiện ngay sau khi import TensorFlow):

```
import tensorflow as tf
# Giả định physical_gpus đã được lấy và physical_gpus[0] tồn tại

tf.config.set_logical_device_configuration(
    physical_gpus[0],
    [tf.config.LogicalDeviceConfiguration(memory_limit=2048),
```



```
tf.config.LogicalDeviceConfiguration(memory_limit=2048)]
)
```

Hai thiết bị logic này được gọi là “/gpu:0” và “/gpu:1”, và bạn có thể sử dụng chúng như thể chúng là hai GPU bình thường. Bạn có thể liệt kê tất cả các thiết bị logic như sau:

```
>>> logical_gpus = tf.config.list_logical_devices("GPU")

>>> logical_gpus
[LogicalDevice(name='/device:GPU:0', device_type='GPU'),
 LogicalDevice(name='/device:GPU:1', device_type='GPU')]
```

Bây giờ hãy xem TensorFlow quyết định thiết bị nào nó nên sử dụng để đặt biến và thực thi các phép toán.

Đặt các Phép toán và Biến trên các Thiết bị

Keras và `tf.data` thường làm rất tốt việc đặt các phép toán và biến vào đúng vị trí, nhưng bạn cũng có thể đặt chúng theo cách thủ công trên từng thiết bị nếu muốn kiểm soát nhiều hơn.

- Thông thường, bạn sẽ muốn đặt các **phép toán tiền xử lý dữ liệu trên CPU** và đặt các **phép toán của mạng nơ-ron trên GPU**.
- GPU thường có băng thông giao tiếp khá hạn chế, vì vậy điều quan trọng là phải **tránh các hoạt động truyền dữ liệu không cần thiết** vào và ra khỏi GPU.
- Việc thêm RAM CPU vào máy khá đơn giản và rẻ, nên thường có rất nhiều. Ngược lại, **RAM của GPU được tích hợp sẵn**: đây là một tài nguyên đắt đỏ và do đó bị hạn chế. Vì vậy, nếu một biến không cần thiết trong vài bước huấn luyện tiếp theo, nó có lẽ nên được đặt trên CPU (ví dụ: các tập dữ liệu thường thuộc về CPU).

Theo mặc định, tất cả các biến và phép toán sẽ được đặt trên **GPU đầu tiên** (có tên là “/gpu:0”), ngoại trừ các biến và phép toán không có kernel GPU; những thứ này được đặt trên CPU (luôn được đặt tên là “/cpu:0”).

Thuộc tính `.device` của một tensor hoặc biến cho bạn biết nó được đặt trên thiết bị nào:

```
>>> a = tf.Variable([1., 2., 3.]) # biến float32 sẽ được đưa lên GPU
>>> a.device
'/job:localhost/replica:0/task:0/device:GPU:0'

>>> b = tf.Variable([1, 2, 3]) # biến int32 sẽ được đưa lên CPU
>>> b.device
'/job:localhost/replica:0/task:0/device:CPU:0'
```

Bạn có thể tạm thời bỏ qua tiền tố `/job:localhost/replica:0/task:0`; chúng ta sẽ thảo luận về các `job`, `replica`, và `task` sau trong chương này. Như bạn thấy, biến đầu tiên được đặt trên GPU #0, là thiết bị mặc định. Tuy nhiên, biến thứ hai được đặt trên CPU: điều này là do **không có kernel GPU cho các biến số nguyên**, hoặc cho các phép toán liên quan đến tensor số nguyên, vì vậy TensorFlow đã chuyển về sử dụng CPU.

Nếu bạn muốn đặt một phép toán trên một thiết bị khác với thiết bị mặc định, hãy sử dụng ngữ cảnh `tf.device()`:

```
>>> with tf.device("/cpu:0"):
...     c = tf.Variable([1., 2., 3.])
...
>>> c.device
'/job:localhost/replica:0/task:0/device:CPU:0'
```

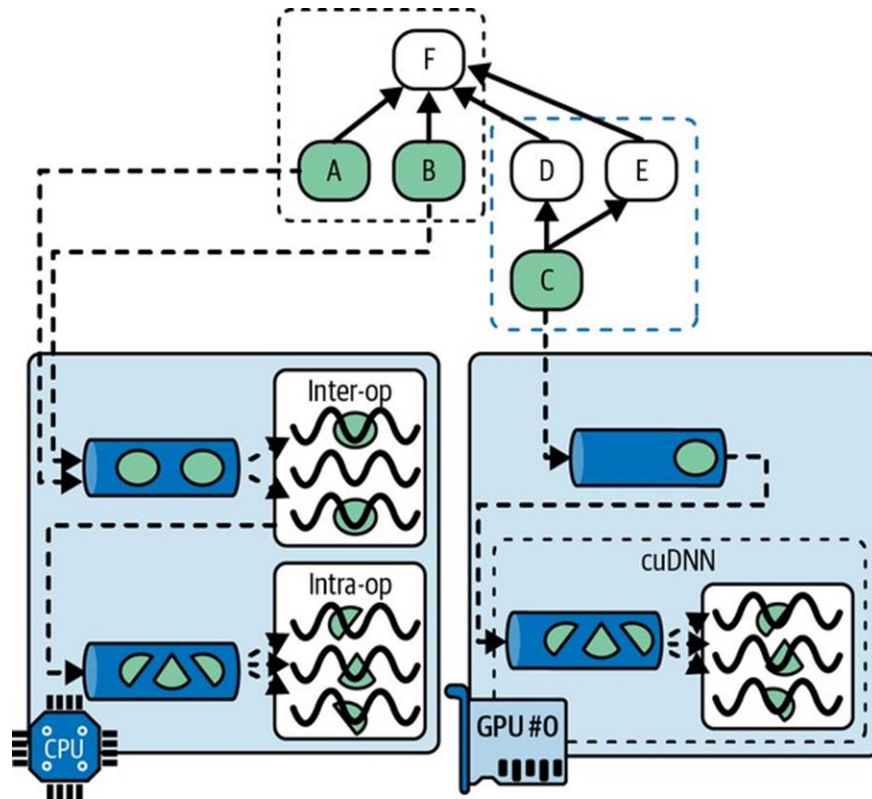
Nếu bạn cố gắng đặt một phép toán hoặc biến một cách tường minh trên một thiết bị không tồn tại hoặc không có kernel tương ứng, TensorFlow sẽ **âm thầm chuyển về thiết bị mà nó sẽ chọn theo mặc định**. Điều này hữu ích khi bạn muốn chạy cùng một đoạn mã trên các máy khác nhau không có cùng số lượng GPU.

Tuy nhiên, bạn có thể chạy `tf.config.set_soft_device_placement(False)` nếu bạn muốn nhận một ngoại lệ (exception) thay thế.

Bây giờ, chính xác thì TensorFlow thực thi các phép toán trên nhiều thiết bị như thế nào?

Thực thi song song trên nhiều thiết bị

Như chúng ta đã thấy trong Chương 12, một trong những lợi ích của việc sử dụng các hàm TF (TF functions) là khả năng song song hóa. Hãy xem xét kỹ hơn một chút. Khi TensorFlow chạy một hàm TF, nó bắt đầu bằng cách phân tích đồ thị của hàm để tìm danh sách các phép toán cần được đánh giá và đếm xem mỗi phép toán có bao nhiêu sự phụ thuộc. Sau đó, TensorFlow thêm mỗi phép toán không có sự phụ thuộc nào (tức là mỗi phép toán nguồn) vào **hàng đợi đánh giá** (evaluation queue) của thiết bị chứa phép toán đó (xem Hình 19-10). Khi một phép toán đã được đánh giá xong, bộ đếm phụ thuộc của mỗi phép toán phụ thuộc vào nó sẽ giảm đi một. Khi bộ đếm phụ thuộc của một phép toán về 0, nó sẽ được đẩy vào hàng đợi đánh giá của thiết bị của nó. Và một khi tất cả các đầu ra đã được tính toán, chúng sẽ được trả về.



Hình 19-10. Thực thi song song một đồ thị TensorFlow

Các phép toán trong hàng đợi đánh giá của CPU được điều phối đến một **nhóm luồng** (thread pool) gọi là **nhóm luồng liên toán tử** (inter-op thread pool). Nếu CPU có nhiều lõi, các phép toán này sẽ được đánh giá song song một cách hiệu quả. Một số phép toán có kernel CPU đa luồng: các kernel này chia nhiệm vụ của chúng thành nhiều phép toán con, được đặt trong một hàng đợi đánh giá khác và được điều phối đến một nhóm luồng thứ hai gọi là **nhóm luồng nội toán tử** (intra-op thread pool) (được chia sẻ bởi tất cả các kernel CPU đa luồng). Tóm lại, nhiều phép toán và phép toán con có thể được đánh giá song song trên các lõi CPU khác nhau.

Đối với GPU, mọi thứ đơn giản hơn một chút. Các phép toán trong hàng đợi đánh giá của GPU được **đánh giá tuần tự**. Tuy nhiên, hầu hết các phép toán đều có kernel GPU đa luồng, thường được triển khai bởi các thư viện mà TensorFlow phụ thuộc vào, chẳng hạn như CUDA và cuDNN. Các triển khai này có các nhóm luồng riêng, và chúng thường khai thác càng nhiều luồng GPU càng tốt (đó là lý do tại sao không cần có nhóm luồng liên toán tử trong GPU: mỗi phép toán đã chiếm gần hết các luồng GPU).

Ví dụ, trong Hình 19-10, các phép toán A, B và C là các phép toán nguồn, vì vậy chúng có thể được đánh giá ngay lập tức. Phép toán A và B được đặt trên CPU, vì vậy chúng được gửi đến hàng đợi đánh giá của CPU, sau đó được điều phối đến nhóm luồng liên toán tử và được đánh giá song song ngay lập tức. Phép toán A tình cờ có một kernel đa luồng; các tính toán của nó được chia thành ba phần, được thực thi song song bởi nhóm luồng nội toán tử. Phép toán C được đưa đến hàng đợi đánh giá của GPU #0, và trong ví dụ này, kernel GPU của nó sử dụng

cuDNN, vốn quản lý nhóm luồng nội toán tử của riêng nó và chạy phép toán trên nhiều luồng GPU song song. Giả sử C hoàn thành trước. Bộ đếm phụ thuộc của D và E được giảm đi và chúng đạt đến 0, vì vậy cả hai phép toán đều được đẩy vào hàng đợi đánh giá của GPU #0 và được thực thi tuần tự.

Lưu ý rằng **C chỉ được đánh giá một lần**, mặc dù cả D và E đều phụ thuộc vào nó. Giả sử B hoàn thành tiếp theo. Bộ đếm phụ thuộc của F giảm từ 4 xuống 3, và vì đó không phải là 0, nó chưa chạy. Khi A, D và E hoàn thành, bộ đếm phụ thuộc của F đạt đến 0, và nó được đẩy vào hàng đợi đánh giá của CPU và được đánh giá. Cuối cùng, TensorFlow trả về các đầu ra được yêu cầu.

Một chút “ma thuật” ✨ khác mà TensorFlow thực hiện là khi hàm TF sửa đổi một tài nguyên có trạng thái, chẳng hạn như một biến: nó đảm bảo rằng **thứ tự thực thi khớp với thứ tự trong mã**, ngay cả khi không có sự phụ thuộc rõ ràng giữa các câu lệnh. Ví dụ, nếu hàm TF của bạn chứa `v.assign_add(1)` theo sau là `v.assign(v * 2)`, TensorFlow sẽ đảm bảo rằng các phép toán này được thực thi theo thứ tự đó.

Với những kiến thức đó, bạn đã có đủ mọi thứ cần thiết để chạy bất kỳ phép toán nào trên bất kỳ thiết bị nào và khai thác sức mạnh của GPU! Dưới đây là một số việc bạn có thể làm:

- Bạn có thể **huấn luyện nhiều mô hình song song**, mỗi mô hình trên một GPU riêng: chỉ cần viết một kịch bản huấn luyện cho mỗi mô hình và chạy chúng song song, đặt `CUDA_DEVICE_ORDER` và `CUDA_VISIBLE_DEVICES` để mỗi kịch bản chỉ thấy một thiết bị GPU duy nhất. Điều này rất tuyệt vời cho việc tinh chỉnh siêu tham số, vì bạn có thể huấn luyện song song nhiều mô hình với các siêu tham số khác nhau.
- Bạn có thể huấn luyện một mô hình trên một GPU và thực hiện **toàn bộ quá trình tiền xử lý song song trên CPU**, sử dụng phương thức `prefetch()` của tập dữ liệu để chuẩn bị trước một vài lô dữ liệu sao cho chúng sẵn sàng khi GPU cần.
- Nếu mô hình của bạn nhận hai hình ảnh làm đầu vào và xử lý chúng bằng **hai CNN** trước khi kết hợp đầu ra của chúng, nó có thể sẽ chạy nhanh hơn nhiều nếu bạn đặt mỗi CNN trên một GPU khác nhau.
- Bạn có thể tạo một **ensemble hiệu quả**: chỉ cần đặt một mô hình đã được huấn luyện khác nhau trên mỗi GPU để bạn có thể nhận được tất cả các dự đoán nhanh hơn nhiều nhằm tạo ra dự đoán cuối cùng của ensemble.

Nhưng nếu bạn muốn tăng tốc độ huấn luyện bằng cách sử dụng nhiều GPU thì sao?

Huấn luyện mô hình trên nhiều thiết bị

Có hai phương pháp chính để huấn luyện một mô hình duy nhất trên nhiều thiết bị: **song song hóa mô hình** (model parallelism), nơi mô hình được chia nhỏ trên các thiết bị, và **song song hóa dữ liệu** (data parallelism), nơi mô hình được sao chép trên mỗi thiết bị, và mỗi bản sao được huấn luyện trên một tập hợp con dữ liệu khác nhau. Hãy cùng xem xét hai lựa chọn này.

Công thức Toán (Phán đoán):

Phần này không chứa các công thức toán học tường minh, mà tập trung vào các khái niệm kỹ thuật và cấu hình phần cứng/phần mềm. Tuy nhiên, có một vài điểm có thể được liên hệ với tính toán:

10. **Tính toán thời gian giảm tốc độ với GPU:** Nếu một tác vụ mất T_{CPU} thời gian trên CPU và T_{GPU} thời gian trên GPU, thì sự tăng tốc (speedup) được tính bằng: $\text{speedup} = \frac{T_{CPU}}{T_{GPU}}$. Văn bản ngụ ý rằng T_{GPU} có thể nhỏ hơn T_{CPU} rất nhiều (từ hàng tuần/ngày xuống hàng giờ/phút).
11. **Phân chia RAM GPU:** Khái niệm chia RAM GPU liên quan đến việc cấp phát tài nguyên. Nếu một GPU có tổng RAM là R_{total} và bạn muốn chia nó cho N chương trình (hoặc thiết bị logic), mỗi chương trình (hoặc thiết bị logic) sẽ nhận được R_{chunk} RAM, thì: $R_{chunk} = \frac{R_{total}}{N}$. Ví dụ, nếu một GPU có 4 GiB RAM và bạn muốn chia nó thành hai thiết bị logic, mỗi thiết bị sẽ có $4 \text{ GiB} / 2 = 2 \text{ GiB}$ RAM. Hoặc nếu bạn muốn giới hạn mỗi chương trình chỉ sử dụng 2 GiB RAM trên mỗi GPU, bạn đang đặt một giới hạn cứng cho việc cấp phát bộ nhớ.

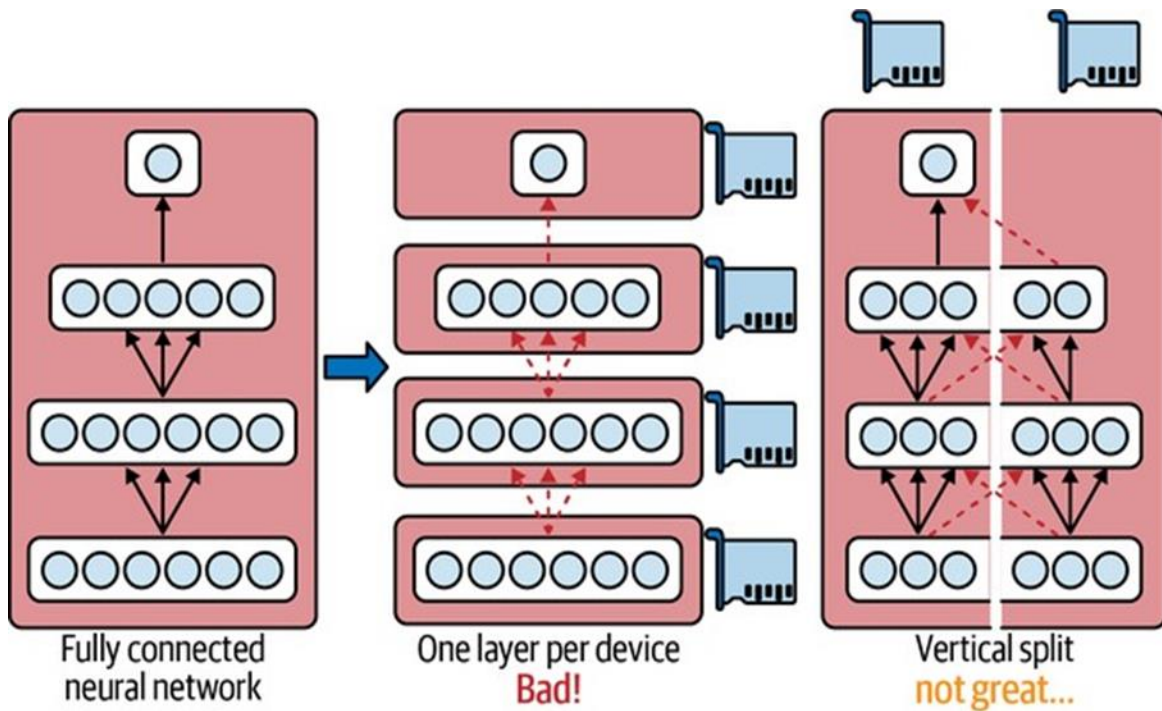
Các phần thảo luận về `tf.config.set_logical_device_configuration` và `tf.config.experimental.set_memory_growth` đều liên quan đến việc kiểm soát cấp phát bộ nhớ, một khía cạnh quan trọng của tối ưu hóa hiệu suất tính toán.

Mô hình song song (Model Parallelism)

Cho đến nay, chúng ta đã huấn luyện mỗi mạng nơ-ron trên một thiết bị duy nhất. Điều gì sẽ xảy ra nếu chúng ta muốn huấn luyện một mạng nơ-ron duy nhất trên nhiều thiết bị?

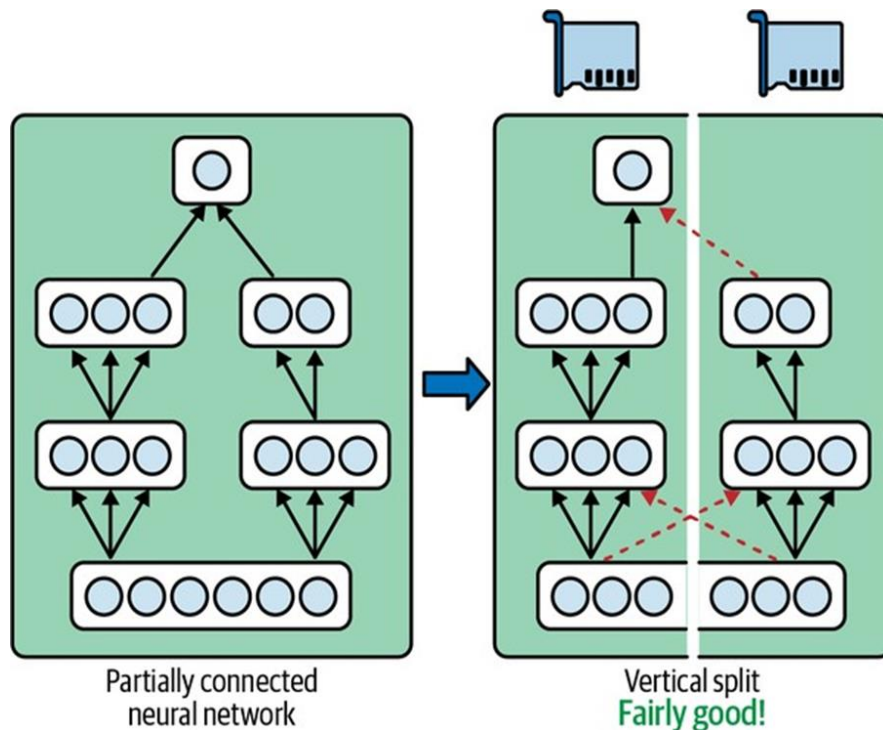
Điều này yêu cầu chia mô hình thành các phần riêng biệt và chạy mỗi phần trên một thiết bị khác nhau. Thật không may, mô hình song song như vậy hóa ra khá phức tạp, và hiệu quả của nó thực sự phụ thuộc vào kiến trúc của mạng nơ-ron của bạn. Đối với các mạng kết nối đầy đủ (fully connected networks), thường không có nhiều lợi ích từ cách tiếp cận này (xem Hình 19-11).

Một cách trực quan, có vẻ như một cách dễ dàng để chia mô hình là đặt mỗi lớp trên một thiết bị khác nhau, nhưng điều này không hoạt động vì mỗi lớp cần đợi đầu ra của lớp trước đó trước khi nó có thể làm bất cứ điều gì. Vậy có lẽ bạn có thể cắt nó theo chiều dọc — ví dụ, với nửa bên trái của mỗi lớp trên một thiết bị, và nửa bên phải trên một thiết bị khác? Điều này tốt hơn một chút, vì cả hai nửa của mỗi lớp thực sự có thể hoạt động song song, nhưng vấn đề là mỗi nửa của lớp tiếp theo yêu cầu đầu ra của cả hai nửa, vì vậy sẽ có rất nhiều giao tiếp giữa các thiết bị (được biểu thị bằng các mũi tên đứt nét). Điều này có khả năng loại bỏ hoàn toàn lợi ích của tính toán song song, vì giao tiếp giữa các thiết bị chậm (và thậm chí còn chậm hơn khi các thiết bị nằm trên các máy khác nhau).



Hình 19-11. Chia một mạng nơ-ron kết nối đầy đủ.

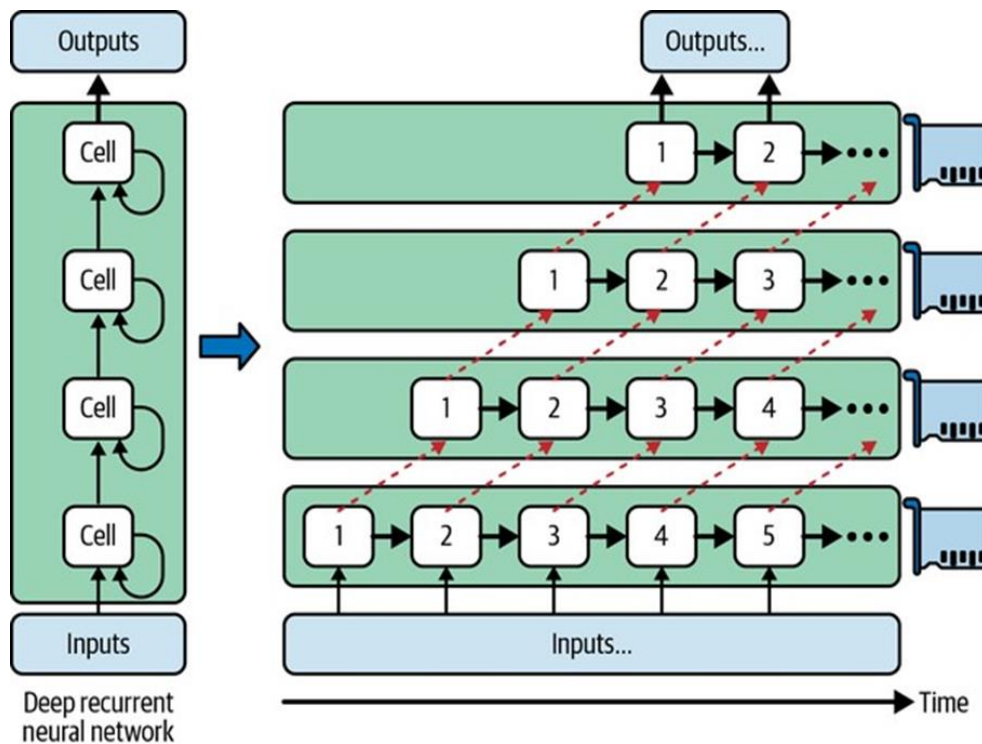
Một số kiến trúc mạng nơ-ron, chẳng hạn như mạng nơ-ron tích chập (convolutional neural networks) (xem Chương 14), chứa các lớp chỉ được kết nối một phần với các lớp thấp hơn, vì vậy việc phân phối các phần trên các thiết bị một cách hiệu quả sẽ dễ dàng hơn nhiều (Hình 19-12).



Hình 19-12. Chia một mạng nơ-ron kết nối một phần.

Các mạng nơ-ron hồi quy sâu (deep recurrent neural networks) (xem Chương 15) có thể được chia hiệu quả hơn một chút trên nhiều GPU. Nếu bạn chia mạng theo chiều ngang bằng cách đặt mỗi lớp trên một thiết bị khác nhau, và cấp dữ liệu đầu vào tuần tự cho mạng để xử lý, thì ở bước thời gian đầu tiên chỉ một thiết bị sẽ hoạt động (xử lý giá trị đầu tiên của chuỗi), ở bước thứ hai hai thiết bị sẽ hoạt động (lớp thứ hai sẽ xử lý đầu ra của lớp đầu tiên cho giá trị đầu tiên, trong khi lớp đầu tiên sẽ xử lý giá trị thứ hai), và đến khi tín hiệu lan truyền đến lớp đầu ra, tất cả các thiết bị sẽ hoạt động đồng thời (Hình 19-13).

Vẫn có rất nhiều giao tiếp giữa các thiết bị đang diễn ra, nhưng vì mỗi ô có thể khá phức tạp, lợi ích của việc chạy nhiều ô song song có thể (về lý thuyết) lớn hơn chi phí giao tiếp. Tuy nhiên, trên thực tế, một chồng các lớp LSTM thông thường chạy trên một GPU duy nhất thực sự chạy nhanh hơn nhiều.



Hình 19-13. Chia một mạng nơ-ron hồi quy sâu.

Tóm lại, mô hình song song có thể tăng tốc việc chạy hoặc huấn luyện một số loại mạng nơ-ron, nhưng không phải tất cả, và nó đòi hỏi sự chăm sóc và điều chỉnh đặc biệt, chẳng hạn như đảm bảo rằng các thiết bị cần giao tiếp nhiều nhất chạy trên cùng một máy. Tiếp theo, chúng ta sẽ xem xét một tùy chọn đơn giản hơn nhiều và thường hiệu quả hơn: song song dữ liệu (data parallelism).

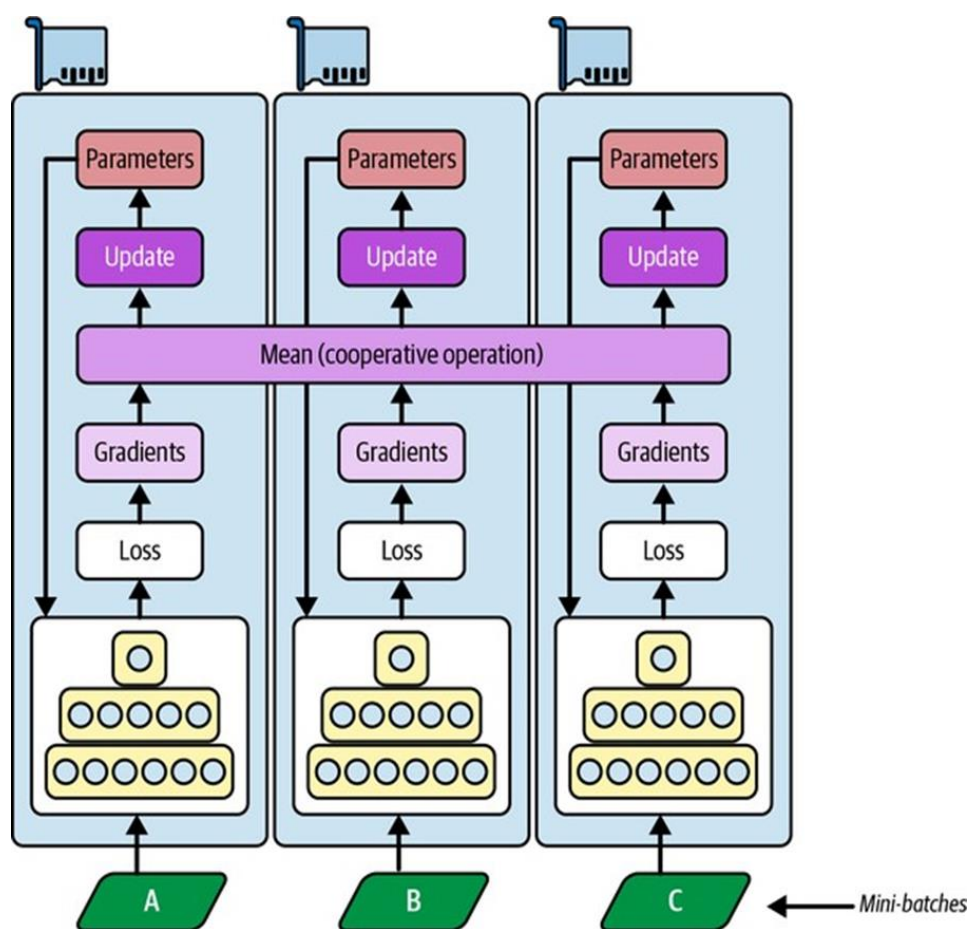
Song song Dữ liệu (Data Parallelism)

Một cách khác để song song hóa quá trình huấn luyện của một mạng nơ-ron là nhân bản nó trên mọi thiết bị và chạy mỗi bước huấn luyện đồng thời trên tất cả các bản sao, sử dụng một

mini-batch khác nhau cho mỗi bản sao. Các gradient được tính toán bởi mỗi bản sao sau đó được tính trung bình, và kết quả được sử dụng để cập nhật các tham số mô hình. Đây được gọi là **song song dữ liệu (data parallelism)**, hoặc đôi khi là **một chương trình, nhiều dữ liệu (single program, multiple data - SPMD)**. Có nhiều biến thể của ý tưởng này, vì vậy hãy xem xét những biến thể quan trọng nhất.

Song song dữ liệu sử dụng chiến lược nhân bản (mirrored strategy)

Cách tiếp cận đơn giản nhất có lẽ là nhân bản hoàn toàn tất cả các tham số mô hình trên tất cả các GPU và luôn áp dụng chính xác các bản cập nhật tham số giống nhau trên mọi GPU. Bằng cách này, tất cả các bản sao luôn giữ nguyên trạng hoàn hảo. Đây được gọi là **chiến lược nhân bản (mirrored strategy)**, và nó hóa ra khá hiệu quả, đặc biệt khi sử dụng một máy duy nhất (xem Hình 19-14).



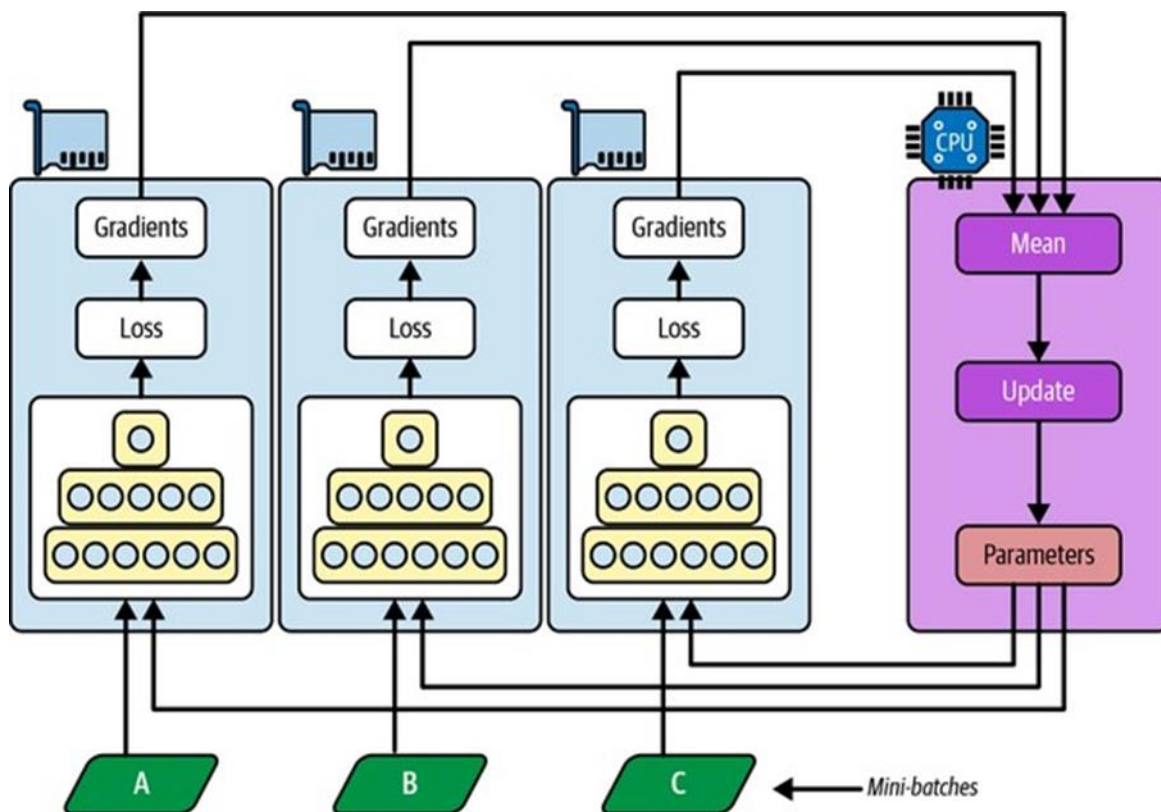
Hình 19-14. Song song dữ liệu sử dụng chiến lược nhân bản.

Phần phức tạp khi sử dụng cách tiếp cận này là tính toán hiệu quả giá trị trung bình của tất cả các gradient từ tất cả các GPU và phân phối kết quả trên tất cả các GPU. Điều này có thể được thực hiện bằng cách sử dụng thuật toán **AllReduce**, một loại thuật toán trong đó nhiều nút hợp tác để thực hiện hiệu quả một phép toán giảm (chẳng hạn như tính trung bình, tổng, và giá trị lớn nhất), đồng thời đảm bảo rằng tất cả các nút đều nhận được cùng một kết quả

cuối cùng. May mắn thay, có các triển khai sẵn có của các thuật toán như vậy, như bạn sẽ thấy.

Song song dữ liệu với tham số tập trung (centralized parameters)

Một cách tiếp cận khác là lưu trữ các tham số mô hình bên ngoài các thiết bị GPU thực hiện các phép tính (được gọi là workers); ví dụ, trên CPU (xem Hình 19-15). Trong một thiết lập phân tán, bạn có thể đặt tất cả các tham số trên một hoặc nhiều máy chủ chỉ có CPU được gọi là **máy chủ tham số (parameter servers)**, mà vai trò duy nhất của chúng là lưu trữ và cập nhật các tham số.



Hình 19-15. Song song dữ liệu với tham số tập trung.

Trong khi chiến lược nhân bản áp đặt cập nhật trọng số đồng bộ trên tất cả các GPU, cách tiếp cận tập trung này cho phép cả cập nhật đồng bộ hoặc bất đồng bộ. Hãy xem xét ưu và nhược điểm của cả hai tùy chọn.

Cập nhật đồng bộ (Synchronous updates)

Với cập nhật đồng bộ, bộ tổng hợp đợi cho đến khi tất cả các gradient có sẵn trước khi nó tính toán gradient trung bình và chuyển chúng cho bộ tối ưu hóa, bộ tối ưu hóa này sẽ cập nhật các tham số mô hình.

Khi một bản sao đã hoàn thành việc tính toán gradient của nó, nó phải đợi các tham số được cập nhật trước khi nó có thể chuyển sang mini-batch tiếp theo. Nhược điểm là một số thiết

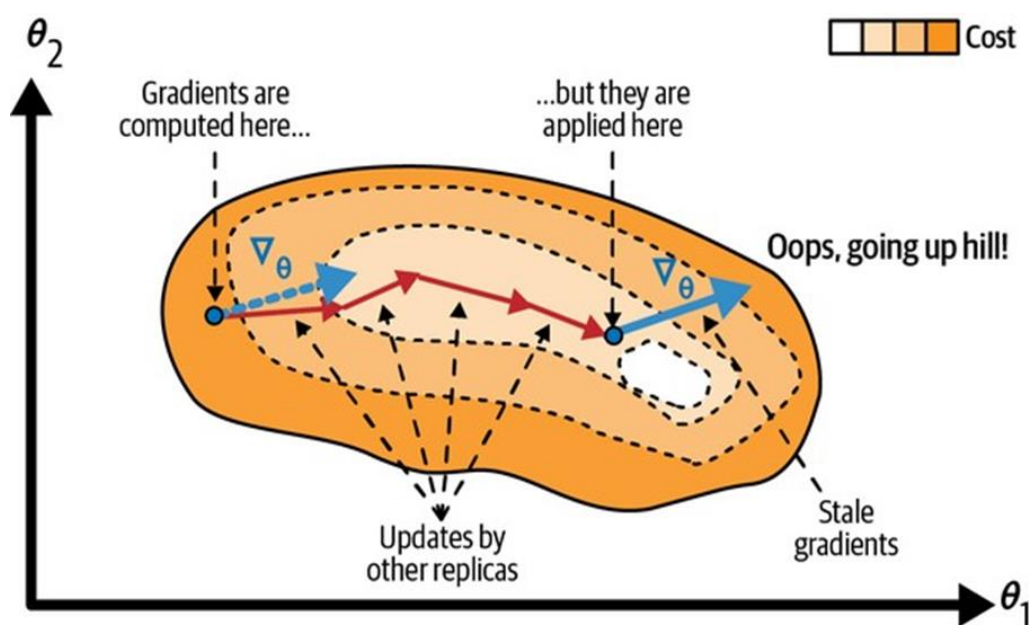
bị có thể chậm hơn các thiết bị khác, vì vậy các thiết bị nhanh sẽ phải đợi các thiết bị chậm ở mỗi bước, làm cho toàn bộ quá trình chậm bằng thiết bị chậm nhất. Hơn nữa, các tham số sẽ được sao chép đến mọi thiết bị gần như cùng lúc (ngay sau khi các gradient được áp dụng), điều này có thể làm bão hòa băng thông của các máy chủ tham số.

Cập nhật bất đồng bộ (Asynchronous updates)

Với cập nhật bất đồng bộ, bất cứ khi nào một bản sao đã hoàn thành việc tính toán gradient, các gradient sẽ ngay lập tức được sử dụng để cập nhật các tham số mô hình. Không có tổng hợp (nó loại bỏ bước “mean” trong Hình 19-15) và không có đồng bộ hóa. Các bản sao hoạt động độc lập với các bản sao khác. Vì không có việc chờ đợi các bản sao khác, cách tiếp cận này chạy nhiều bước huấn luyện hơn mỗi phút. Hơn nữa, mặc dù các tham số vẫn cần được sao chép đến mọi thiết bị ở mỗi bước, điều này xảy ra vào các thời điểm khác nhau đối với mỗi bản sao, vì vậy rủi ro bão hòa băng thông giảm.

Song song dữ liệu với cập nhật bất đồng bộ là một lựa chọn hấp dẫn vì sự đơn giản của nó, không có độ trễ đồng bộ hóa và sử dụng băng thông tốt hơn. Tuy nhiên, mặc dù nó hoạt động khá tốt trên thực tế, nhưng gần như đáng ngạc nhiên khi nó hoạt động được! Thật vậy, vào thời điểm một bản sao đã hoàn thành việc tính toán gradient dựa trên một số giá trị tham số, các tham số này sẽ đã được cập nhật nhiều lần bởi các bản sao khác (trung bình $N - 1$ lần, nếu có N bản sao), và không có gì đảm bảo rằng các gradient đã tính toán vẫn sẽ trở đúng hướng (xem Hình 19-16).

Khi các gradient bị lỗi thời nghiêm trọng, chúng được gọi là **gradient lỗi thời (stale gradients)**: chúng có thể làm chậm quá trình hội tụ, gây ra nhiễu và hiệu ứng lung lay (đường cong học tập có thể chứa các dao động tạm thời), hoặc chúng thậm chí có thể làm cho thuật toán huấn luyện phân kỳ.



Hình 19-16. Gradient lỗi thời khi sử dụng cập nhật bất đồng bộ.

Có một vài cách bạn có thể giảm ảnh hưởng của gradient lỗi thời:

- Giảm tốc độ học (learning rate).
- Bỏ qua các gradient lỗi thời hoặc giảm tỷ lệ của chúng.
- Điều chỉnh kích thước mini-batch.
- Bắt đầu vài epoch đầu tiên chỉ sử dụng một bản sao (đây được gọi là **giai đoạn khởi động - warmup phase**). Gradient lỗi thời có xu hướng gây hại nhiều hơn vào đầu quá trình huấn luyện, khi gradient thường lớn và các tham số chưa ổn định vào một thung lũng của hàm chi phí, vì vậy các bản sao khác nhau có thể đẩy các tham số theo những hướng khá khác nhau.

Một bài báo được xuất bản bởi nhóm Google Brain vào năm 2016 đã so sánh các cách tiếp cận khác nhau và thấy rằng việc sử dụng cập nhật đồng bộ với một vài bản sao dự phòng hiệu quả hơn so với việc sử dụng cập nhật bất đồng bộ, không chỉ hội tụ nhanh hơn mà còn tạo ra một mô hình tốt hơn. Tuy nhiên, đây vẫn là một lĩnh vực nghiên cứu tích cực, vì vậy bạn không nên loại bỏ cập nhật bất đồng bộ ngay lập tức.

Bão hòa băng thông (Bandwidth saturation)

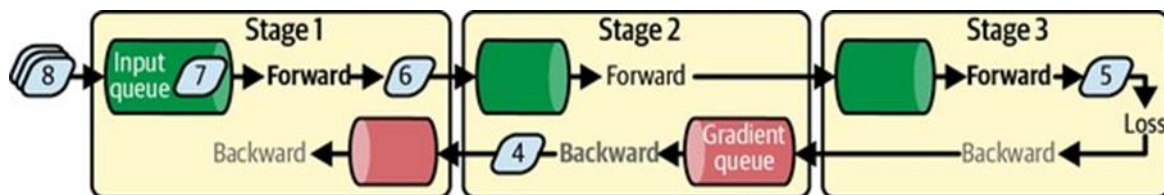
Cho dù bạn sử dụng cập nhật đồng bộ hay bất đồng bộ, song song dữ liệu với các tham số tập trung vẫn yêu cầu truyền tham số mô hình từ máy chủ tham số đến mọi bản sao ở đầu mỗi bước huấn luyện, và các gradient theo hướng ngược lại ở cuối mỗi bước huấn luyện. Tương tự, khi sử dụng chiến lược nhân bản, các gradient được tạo ra bởi mỗi GPU sẽ cần được chia sẻ với mọi GPU khác. Thật không may, thường có một điểm mà việc thêm một GPU bổ sung sẽ không cải thiện hiệu suất chút nào vì thời gian dành cho việc di chuyển dữ liệu vào và ra khỏi RAM GPU (và qua mạng trong một thiết lập phân tán) sẽ lớn hơn lợi ích tăng tốc đạt được bằng cách chia tải tính toán. Tại thời điểm đó, việc thêm nhiều GPU hơn sẽ chỉ làm trầm trọng thêm tình trạng bão hòa băng thông và thực sự làm chậm quá trình huấn luyện.

Bão hòa nghiêm trọng hơn đối với các mô hình dày đặc lớn (large dense models), vì chúng có nhiều tham số và gradient để truyền. Nó ít nghiêm trọng hơn đối với các mô hình nhỏ (nhưng lợi ích song song hóa bị hạn chế) và đối với các mô hình thưa lớn (large sparse models), nơi các gradient thường chủ yếu là số 0 và do đó có thể được truyền thông hiệu quả. Jeff Dean, người khởi xướng và lãnh đạo dự án Google Brain, đã báo cáo tốc độ tăng tốc điển hình là 25–40 lần khi phân tán các phép tính trên 50 GPU cho các mô hình dày đặc, và tốc độ tăng tốc 300 lần cho các mô hình thưa hơn được huấn luyện trên 500 GPU. Như bạn có thể thấy, các mô hình thưa thực sự mở rộng tốt hơn. Dưới đây là một vài ví dụ cụ thể:

- Dịch máy nơ-ron: Tăng tốc 6 lần trên 8 GPU
- Inception/ImageNet: Tăng tốc 32 lần trên 50 GPU
- RankBrain: Tăng tốc 300 lần trên 500 GPU

Có rất nhiều nghiên cứu đang diễn ra để giảm thiểu vấn đề bão hòa băng thông, với mục tiêu cho phép huấn luyện mở rộng tuyến tính với số lượng GPU có sẵn. Ví dụ, một bài báo năm

2018 của một nhóm các nhà nghiên cứu từ Đại học Carnegie Mellon, Đại học Stanford và Microsoft Research đã đề xuất một hệ thống có tên PipeDream đã giảm hơn 90% giao tiếp mạng, giúp có thể huấn luyện các mô hình lớn trên nhiều máy. Họ đã đạt được điều này bằng cách sử dụng một kỹ thuật mới được gọi là **song song đường ống (pipeline parallelism)**, kết hợp song song mô hình và song song dữ liệu: mô hình được chia thành các phần liên tiếp, được gọi là các **giai đoạn (stages)**, mỗi giai đoạn được huấn luyện trên một máy khác nhau. Điều này tạo ra một đường ống bất đồng bộ trong đó tất cả các máy hoạt động song song với rất ít thời gian nhàn rỗi. Trong quá trình huấn luyện, mỗi giai đoạn luân phiên một vòng truyền xuôi (forward propagation) và một vòng truyền ngược (backpropagation) (xem Hình 19-17): nó lấy một mini-batch từ hàng đợi đầu vào của nó, xử lý nó, và gửi đầu ra đến hàng đợi đầu vào của giai đoạn tiếp theo, sau đó nó lấy một mini-batch gradient từ hàng đợi gradient của nó, truyền ngược các gradient này và cập nhật các tham số mô hình của chính nó, và đẩy các gradient đã truyền ngược đến hàng đợi gradient của giai đoạn trước đó. Sau đó, nó lặp lại toàn bộ quá trình. Mỗi giai đoạn cũng có thể sử dụng song song dữ liệu thông thường (ví dụ: sử dụng chiến lược nhân bản), độc lập với các giai đoạn khác.



Hình 19-17. Song song đường ống của PipeDream.

Tuy nhiên, như được trình bày ở đây, PipeDream sẽ không hoạt động tốt. Để hiểu tại sao, hãy xem xét mini-batch #5 trong Hình 19-17: khi nó đi qua giai đoạn 1 trong quá trình truyền xuôi, các gradient từ mini-batch #4 vẫn chưa được truyền ngược qua giai đoạn đó, nhưng đến khi gradient của #5 chảy ngược về giai đoạn 1, gradient của #4 sẽ đã được sử dụng để cập nhật các tham số mô hình, vì vậy gradient của #5 sẽ hơi lỗi thời. Như chúng ta đã thấy, điều này có thể làm giảm tốc độ huấn luyện và độ chính xác, và thậm chí làm cho nó phân kỳ: càng có nhiều giai đoạn, vấn đề này càng trở nên tồi tệ hơn. Các tác giả của bài báo đã đề xuất các phương pháp để giảm thiểu vấn đề này, tuy nhiên: ví dụ, mỗi giai đoạn lưu trọng số trong quá trình truyền xuôi và khôi phục chúng trong quá trình truyền ngược, để đảm bảo rằng cùng một trọng số được sử dụng cho cả truyền xuôi và truyền ngược. Điều này được gọi là **cất giữ trọng số (weight stashing)**.

Nhờ điều này, PipeDream thể hiện khả năng mở rộng ấn tượng, vượt xa song song dữ liệu đơn giản.

Bước đột phá mới nhất trong lĩnh vực nghiên cứu này đã được xuất bản trong một bài báo năm 2022 bởi các nhà nghiên cứu của Google: họ đã phát triển một hệ thống có tên Pathways sử dụng song song mô hình tự động, lập lịch nhóm bất đồng bộ, và các kỹ thuật khác để đạt được gần 100% sử dụng phần cứng trên hàng ngàn TPU! **Lập lịch (scheduling)** có nghĩa là tổ chức khi nào và ở đâu mỗi tác vụ phải chạy, và **lập lịch nhóm (gang scheduling)** có nghĩa là chạy các tác vụ liên quan cùng lúc song song và gần nhau để giảm thời gian các tác vụ phải đợi đầu ra của các tác vụ khác. Như chúng ta đã thấy trong Chương

16, hệ thống này đã được sử dụng để huấn luyện một mô hình ngôn ngữ khổng lồ trên hơn 6.000 TPU, với gần 100% sử dụng phần cứng: đó là một kỳ tích kỹ thuật đáng kinh ngạc.

Tại thời điểm viết bài này, Pathways chưa công khai, nhưng có khả năng trong tương lai gần bạn sẽ có thể huấn luyện các mô hình lớn trên Vertex AI bằng cách sử dụng Pathways hoặc một hệ thống tương tự. Trong thời gian chờ đợi, để giảm vấn đề bão hòa, bạn có thể sẽ muốn sử dụng một vài GPU mạnh mẽ hơn là nhiều GPU yếu, và nếu bạn cần huấn luyện một mô hình trên nhiều máy chủ, bạn nên nhóm các GPU của mình trên ít máy chủ và có kết nối rất tốt. Bạn cũng có thể thử giảm độ chính xác dấu phẩy động từ 32 bit (tf.float32) xuống 16 bit (tf.bfloat16). Điều này sẽ giảm một nửa lượng dữ liệu cần truyền, thường không ảnh hưởng nhiều đến tốc độ hội tụ hoặc hiệu suất của mô hình. Cuối cùng, nếu bạn đang sử dụng các tham số tập trung, bạn có thể **chia nhỏ (shard)** các tham số trên nhiều máy chủ tham số: việc thêm nhiều máy chủ tham số sẽ giảm tải mạng trên mỗi máy chủ và hạn chế rủi ro bão hòa băng thông.

Được rồi, bây giờ chúng ta đã đi qua tất cả lý thuyết, hãy thực sự huấn luyện một mô hình trên nhiều GPU!

Phán đoán và Công thức Toán:

Trong phần này, các công thức toán học không được trình bày rõ ràng nhưng các khái niệm về hiệu suất và tính toán được thảo luận sâu. Dưới đây là những điểm có thể liên hệ với toán học:

12. Hiệu quả của Model Parallelism và Data Parallelism: Cả hai phương pháp đều nhằm mục đích giảm thời gian huấn luyện. Nếu thời gian huấn luyện trên một thiết bị là T_1 , và trên N thiết bị là T_N . Mục tiêu là $T_N \ll T_1$.

- **Speedup:** Một thước đo hiệu suất là “speedup”, được định nghĩa là $S_N = T_1/T_N$. Mục tiêu là đạt được speedup càng gần N càng tốt (linear speedup).
- **Hiệu quả (Efficiency):** $E_N = S_N/N$. Mục tiêu là $E_N \approx 1$. Văn bản cho thấy rằng trong thực tế, E_N thường nhỏ hơn 1 do chi phí giao tiếp và độ trễ.

13. Chi phí Giao tiếp (Communication Overhead): Chi phí giao tiếp giữa các thiết bị hoặc máy chủ là một yếu tố quan trọng làm giảm hiệu quả song song. Thời gian dành cho giao tiếp có thể được xem là T_{comm} .

$$T_N = T_{comp} + T_{comm}$$

Trong đó T_{comp} là thời gian tính toán thực tế. Khi N tăng, T_{comp} có thể giảm, nhưng T_{comm} thường tăng, dẫn đến điểm bão hòa (saturation point) nơi việc thêm GPU không còn cải thiện hiệu suất, thậm chí làm tệ hơn.

14. Tác động của Stale Gradients: Trong cập nhật bất đồng bộ, gradient từ một bản sao có thể bị “lỗi thời” khi áp dụng do các bản sao khác đã cập nhật tham số. Nếu có N bản sao, trung bình một gradient sẽ lỗi thời $N - 1$ lần cập nhật của các bản sao khác. Điều này có thể được hình dung bằng một hàm lỗi (loss function) $L(\theta)$ và việc cập nhật tham số θ theo gradient $-\nabla L(\theta)$. Gradient lỗi thời có thể không còn chỉ hướng

dốc nhất đến cực tiểu, dẫn đến đường đi cập nhật “lung lay” (oscillations) hoặc thậm chí phân kỳ.

$$\theta_{t+1} = \theta_t - \eta \cdot \nabla L(\theta_{t-\delta})$$

Trong đó δ là độ trễ (staleness) của gradient.

- 15. Lợi ích của Half-Precision Floats (tf.bfloat16):** Việc chuyển từ tf.float32 (32 bit) sang tf.bfloat16 (16 bit) giảm kích thước dữ liệu cần truyền đi một nửa (factor of 2). Điều này trực tiếp giảm T_{comm} , giúp giảm bảo hòa băng thông.

$$Kí'chthứcdữliệumới = \frac{16}{32} \times Kí'chthứcdữliệugo~c = 0.5 \times Kí'chthứcdữliệugo~c$$

- 16. Sharding Parameters:** Khi chia nhỏ tham số (sharding), tải mạng trên mỗi máy chủ tham số được phân chia. Nếu tổng tải mạng là L_{total} và bạn chia nhỏ trên M máy chủ tham số, thì tải trung bình trên mỗi máy chủ là L_{total}/M . Điều này giúp phân tán tải và tránh tắc nghẽn.

Các con số về tốc độ tăng tốc (ví dụ: 25-40 lần trên 50 GPU, 300 lần trên 500 GPU) là các giá trị thực nghiệm, cho thấy hiệu quả của song song hóa trong các kịch bản thực tế.

Huấn luyện Quy mô lớn bằng API Chiến lược Phân phối (Distribution Strategies API)

May mắn thay, TensorFlow đi kèm với một API rất tuyệt vời, giúp xử lý tất cả sự phức tạp của việc phân phối mô hình của bạn trên nhiều thiết bị và máy: API chiến lược phân phối. Để huấn luyện một mô hình Keras trên tất cả các GPU khả dụng (trên một máy duy nhất, hiện tại) bằng cách sử dụng song song dữ liệu với chiến lược nhân bản (mirrored strategy), chỉ cần tạo một đối tượng MirroredStrategy, gọi phương thức scope() của nó để có được một ngữ cảnh phân phối, và gói việc tạo và biên dịch mô hình của bạn bên trong ngữ cảnh đó. Sau đó, gọi phương thức fit() của mô hình như bình thường:

```
import tensorflow as tf

# Giả định X_train, y_train, X_valid, y_valid đã được tải và chia
# Ví dụ: (X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
# X_train = X_train.astype('float32') / 255.0
# X_valid = X_valid.astype('float32') / 255.0
# y_train = tf.keras.utils.to_categorical(y_train, num_classes=10)
# y_valid = tf.keras.utils.to_categorical(y_valid, num_classes=10)

strategy = tf.distribute.MirroredStrategy()

with strategy.scope():
    model = tf.keras.Sequential([ # tạo một mô hình Keras như bình thường
        tf.keras.layers.Flatten(input_shape=(28, 28)),
        tf.keras.layers.Dense(128, activation='relu'),
```

```

        tf.keras.layers.Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=
['accuracy']) # biên dịch mô hình như bình thường

batch_size = 100 # tốt nhất nên chia hết cho số lượng bản sao
model.fit(X_train, y_train, epochs=10, validation_data=(X_valid, y_valid),
        batch_size=batch_size)

```

Dưới lớp vỏ, Keras nhận thức được phân phối, vì vậy trong ngữ cảnh `MirroredStrategy` này, nó biết rằng nó phải nhân bản tất cả các biến và phép toán trên tất cả các thiết bị GPU khả dụng. Nếu bạn nhìn vào trọng số của mô hình, chúng có kiểu `MirroredVariable`:

```

>>> type(model.weights[0])
<class 'tensorflow.python.distribute.values.MirroredVariable'>

```

Lưu ý rằng phương thức `fit()` sẽ tự động chia mỗi lô huấn luyện trên tất cả các bản sao, vì vậy tốt hơn là đảm bảo rằng kích thước lô chia hết cho số lượng bản sao (tức là số lượng GPU khả dụng) để tất cả các bản sao nhận được các lô có cùng kích thước. Và thế là xong! Quá trình huấn luyện nói chung sẽ nhanh hơn đáng kể so với việc sử dụng một thiết bị, và thay đổi mã là rất nhỏ.

Khi bạn đã hoàn thành việc huấn luyện mô hình của mình, bạn có thể sử dụng nó để đưa ra dự đoán một cách hiệu quả: gọi phương thức `predict()`, và nó sẽ tự động chia lô trên tất cả các bản sao, thực hiện dự đoán song song. Một lần nữa, kích thước lô phải chia hết cho số lượng bản sao. Nếu bạn gọi phương thức `save()` của mô hình, nó sẽ được lưu dưới dạng một mô hình thông thường, không phải là một mô hình được nhân bản với nhiều bản sao. Vì vậy, khi bạn tải nó, nó sẽ chạy như một mô hình thông thường, trên một thiết bị duy nhất: theo mặc định trên GPU #0, hoặc trên CPU nếu không có GPU. Nếu bạn muốn tải một mô hình và chạy nó trên tất cả các thiết bị khả dụng, bạn phải gọi `tf.keras.models.load_model()` trong một ngữ cảnh phân phối:

```

# Giả định strategy đã được định nghĩa và "my_mirrored_model" là đường dẫn đ
n mô hình đã lưu
with strategy.scope():
    model = tf.keras.models.load_model("my_mirrored_model")

```

Nếu bạn chỉ muốn sử dụng một tập hợp con của tất cả các thiết bị GPU khả dụng, bạn có thể truyền danh sách đó cho hàm tạo của `MirroredStrategy`:

```

strategy = tf.distribute.MirroredStrategy(devices=["/gpu:0", "/gpu:1"])

```

Theo mặc định, lớp `MirroredStrategy` sử dụng NVIDIA Collective Communications Library (NCCL) cho phép toán AllReduce trung bình, nhưng bạn có thể thay đổi nó bằng cách đặt đối số `cross_device_ops` thành một instance của lớp `tf.distribute.HierarchicalCopyAllReduce`, hoặc một instance của lớp `tf.distribute.ReductionToOneDevice`. Tùy chọn NCCL mặc định dựa trên lớp

`tf.distribute.NcclAllReduce`, thường nhanh hơn, nhưng điều này phụ thuộc vào số lượng và loại GPU, vì vậy bạn có thể muốn thử các lựa chọn thay thế.

Nếu bạn muốn thử sử dụng song song dữ liệu với các tham số tập trung, hãy thay thế `MirroredStrategy` bằng `CentralStorageStrategy`:

```
strategy = tf.distribute.experimental.CentralStorageStrategy()
```

Bạn có thể tùy chọn đặt đối số `compute_devices` để chỉ định danh sách các thiết bị bạn muốn sử dụng làm workers — theo mặc định nó sẽ sử dụng tất cả các GPU khả dụng — và bạn có thể tùy chọn đặt đối số `parameter_device` để chỉ định thiết bị bạn muốn lưu trữ các tham số. Theo mặc định nó sẽ sử dụng CPU, hoặc GPU nếu chỉ có một.

Bây giờ hãy xem cách huấn luyện một mô hình trên một cụm máy chủ TensorFlow!

Huấn luyện Mô hình trên Cụm TensorFlow

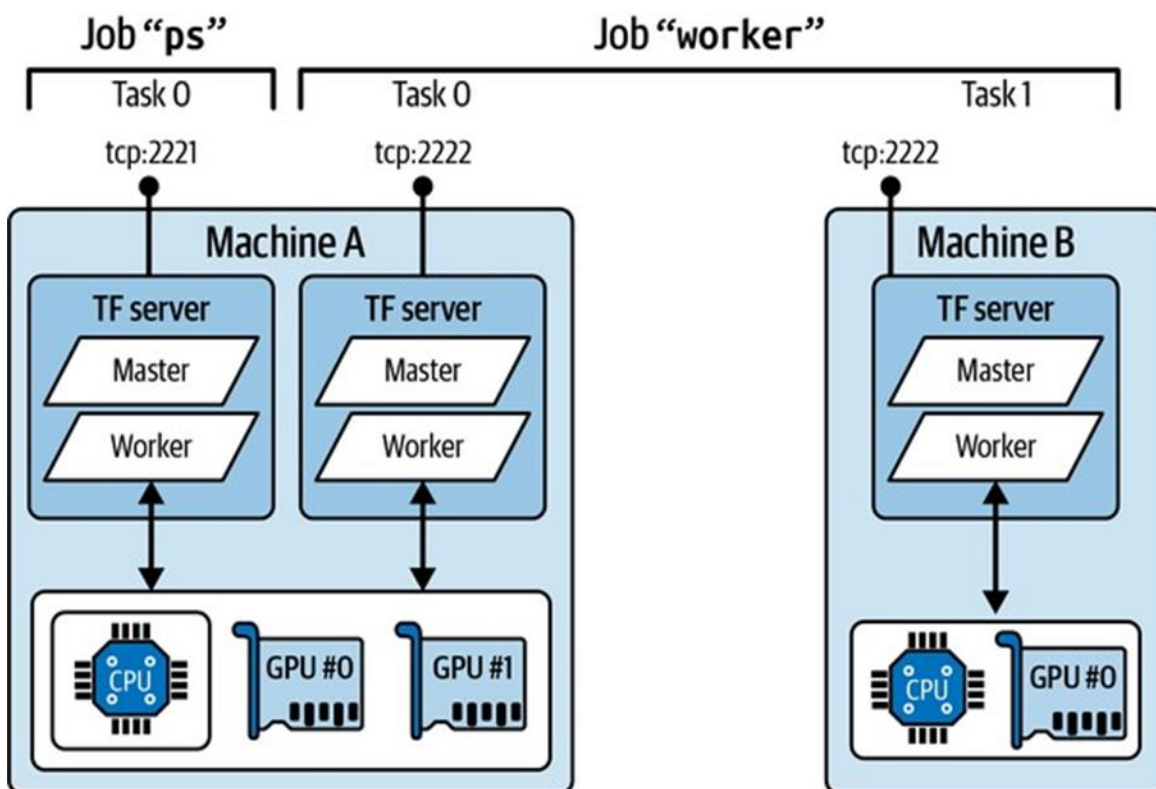
Một cụm TensorFlow là một nhóm các tiến trình TensorFlow chạy song song, thường trên các máy khác nhau, và giao tiếp với nhau để hoàn thành một công việc — ví dụ, huấn luyện hoặc thực thi một mô hình mạng nơ-ron. Mỗi tiến trình TF trong cụm được gọi là một **tác vụ (task)**, hoặc một **máy chủ TF (TF server)**. Nó có một địa chỉ IP, một cổng, và một loại (còn được gọi là vai trò hoặc công việc của nó). Loại này có thể là “worker”, “chief”, “ps” (parameter server), hoặc “evaluator”:

- Mỗi **worker** thực hiện các phép tính, thường trên một máy có một hoặc nhiều GPU.
- **Chief** cũng thực hiện các phép tính (nó là một worker), nhưng nó cũng xử lý các công việc bổ sung như ghi nhật ký TensorBoard hoặc lưu điểm kiểm tra (checkpoints). Có một chief duy nhất trong một cụm. Nếu không có chief nào được chỉ định rõ ràng, thì theo quy ước, worker đầu tiên là chief.
- Một **parameter server** chỉ theo dõi các giá trị biến, và nó thường nằm trên một máy chỉ có CPU. Loại tác vụ này chỉ được sử dụng với `ParameterServerStrategy`.
- Một **evaluator** rõ ràng là chịu trách nhiệm đánh giá. Loại này không được sử dụng thường xuyên, và khi được sử dụng, thường chỉ có một evaluator.

Để khởi động một cụm TensorFlow, bạn phải xác định đặc tả của nó. Điều này có nghĩa là xác định địa chỉ IP, cổng TCP và loại của mỗi tác vụ. Ví dụ, đặc tả cụm sau đây định nghĩa một cụm với ba tác vụ (hai worker và một parameter server; xem Hình 19-18). Đặc tả cụm là một dictionary với một khóa cho mỗi công việc, và các giá trị là danh sách các địa chỉ tác vụ (IP:cổng):

```
cluster_spec = {
    "worker": [
        "machine-a.example.com:2222", # /job:worker/task:0
        "machine-b.example.com:2222"  # /job:worker/task:1
    ],
    "ps": ["machine-a.example.com:2221"] # /job:ps/task:0
}
```


Nói chung sẽ có một tác vụ duy nhất cho mỗi máy, nhưng như ví dụ này cho thấy, bạn có thể cấu hình nhiều tác vụ trên cùng một máy nếu muốn. Trong trường hợp này, nếu chúng chia sẻ cùng GPU, hãy đảm bảo RAM được chia thích hợp, như đã thảo luận trước đó.



Hình 19-18. Một ví dụ về cụm TensorFlow.

Khi bạn khởi động một tác vụ, bạn phải cung cấp cho nó đặc tả cụm, và bạn cũng phải cho nó biết loại và chỉ mục của nó (ví dụ: worker #0). Cách đơn giản nhất để chỉ định mọi thứ cùng một lúc (cả đặc tả cụm và loại và chỉ mục của tác vụ hiện tại) là đặt biến môi trường `TF_CONFIG` trước khi khởi động TensorFlow. Nó phải là một dictionary được mã hóa JSON chứa một đặc tả cụm (dưới khóa “cluster”) và loại và chỉ mục của tác vụ hiện tại (dưới khóa “task”). Ví dụ, biến môi trường `TF_CONFIG` sau đây sử dụng cụm chúng ta vừa định nghĩa và chỉ định rằng tác vụ sẽ khởi động là worker #0:

```
import os
import json

os.environ["TF_CONFIG"] = json.dumps({
    "cluster": cluster_spec,
    "task": {"type": "worker", "index": 0}
})
```

Bây giờ hãy huấn luyện một mô hình trên một cụm! Chúng ta sẽ bắt đầu với chiến lược nhân bản (mirrored strategy). Đầu tiên, bạn cần đặt biến môi trường `TF_CONFIG` thích hợp cho mỗi tác vụ. Sẽ không có máy chủ tham số (xóa khóa “ps” trong đặc tả cụm), và nói chung bạn sẽ

muốn một worker duy nhất cho mỗi máy. Đảm bảo đặt một chỉ mục tác vụ khác nhau cho mỗi tác vụ. Cuối cùng, chạy script sau trên mỗi worker:

```
import tempfile
import tensorflow as tf
# Giả định X_train, y_train, X_valid, y_valid đã được tải và chia
# Giả định os.environ["TF_CONFIG"] đã được thiết lập đúng cho từng tác vụ

strategy = tf.distribute.MultiWorkerMirroredStrategy() # ở phần đầu!
resolver = tf.distribute.cluster_resolver.TFConfigClusterResolver()
print(f"Starting task {resolver.task_type} #{resolver.task_id}")

# ... Tải và chia tập dữ liệu MNIST (các đoạn mã từ trước) ...
# Ví dụ đơn giản:
(X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
X_train = X_train.astype('float32') / 255.0
X_valid = X_valid.astype('float32') / 255.0 # Bạn có thể cần tạo X_valid/y_val
lid từ X_train/y_train
y_train = tf.keras.utils.to_categorical(y_train, num_classes=10)
y_valid = tf.keras.utils.to_categorical(y_valid, num_classes=10)

with strategy.scope():
    model = tf.keras.Sequential([ # xây dựng mô hình Keras
        tf.keras.layers.Flatten(input_shape=(28, 28)),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(10, activation='softmax')
    ])
    model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=
['accuracy']) # biên dịch mô hình

model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=10)

if resolver.task_id == 0: # chief lưu mô hình vào đúng vị trí
    model.save("my_mnist_multiworker_model", save_format="tf")
else:
    tmpdir = tempfile.mkdtemp() # các worker khác lưu vào thư mục tạm thời
    model.save(tmpdir, save_format="tf")
    tf.io.gfile.rmtree(tmpdir) # và chúng ta có thể xóa thư mục này ở cuối!
```

Đó gần như là cùng một mã bạn đã sử dụng trước đó, ngoại trừ lần này bạn đang sử dụng `MultiWorkerMirroredStrategy`. Khi bạn khởi động script này trên các worker đầu tiên, chúng sẽ bị chặn ở bước `AllReduce`, nhưng quá trình huấn luyện sẽ bắt đầu ngay khi worker cuối cùng khởi động, và bạn sẽ thấy tất cả chúng cùng tiến độ ở cùng một tốc độ chính xác vì chúng đồng bộ hóa ở mỗi bước.

Có hai triển khai `AllReduce` cho chiến lược phân phối này: một thuật toán `AllReduce` vòng dựa trên gRPC cho giao tiếp mạng, và triển khai của NCCL. Thuật toán tốt nhất để sử dụng phụ thuộc vào số lượng worker, số lượng và loại GPU, và mạng. Theo mặc định, TensorFlow

sẽ áp dụng một số phương pháp heuristic để chọn thuật toán phù hợp cho bạn, nhưng bạn có thể buộc NCCL (hoặc RING) như sau:

```
strategy = tf.distribute.MultiWorkerMirroredStrategy(  
    communication_options=tf.distribute.experimental.CommunicationOptions(  
        implementation=tf.distribute.experimental.CollectiveCommunication.NCCL  
    )  
)
```

Nếu bạn muốn triển khai song song dữ liệu bất đồng bộ với các máy chủ tham số, hãy thay đổi chiến lược thành `ParameterServerStrategy`, thêm một hoặc nhiều máy chủ tham số, và cấu hình `TF_CONFIG` thích hợp cho mỗi tác vụ. Lưu ý rằng mặc dù các worker sẽ hoạt động bất đồng bộ, các bản sao trên mỗi worker sẽ hoạt động đồng bộ.

Cuối cùng, nếu bạn có quyền truy cập vào TPU trên Google Cloud — ví dụ, nếu bạn sử dụng Colab và bạn đặt loại bộ tăng tốc thành TPU — thì bạn có thể tạo một `TPUStrategy` như sau:

```
import tensorflow as tf  
  
# Điều này cần được chạy ngay sau khi import TensorFlow.  
resolver = tf.distribute.cluster_resolver.TPUClusterResolver()  
tf.tpu.experimental.initialize_tpu_system(resolver)  
strategy = tf.distribute.experimental.TPUStrategy(resolver)  
# Sau đó, bạn có thể sử dụng chiến lược này như bình thường.
```

Bây giờ bạn đã có thể huấn luyện các mô hình trên nhiều GPU và nhiều máy chủ: hãy tự thưởng cho mình! Tuy nhiên, nếu bạn muốn huấn luyện một mô hình rất lớn, bạn sẽ cần nhiều GPU, trên nhiều máy chủ, điều này sẽ yêu cầu hoặc mua rất nhiều phần cứng hoặc quản lý rất nhiều máy ảo đám mây. Trong nhiều trường hợp, việc sử dụng dịch vụ đám mây chuyên lo việc cấp phát và quản lý tất cả cơ sở hạ tầng này cho bạn, ngay khi bạn cần, sẽ ít rắc rối và ít tốn kém hơn. Hãy xem cách thực hiện điều đó bằng cách sử dụng Vertex AI.

Phán đoán và Công thức Toán:

Trong phần này, các công thức toán học không được trình bày rõ ràng, nhưng các khái niệm về huấn luyện phân tán và tối ưu hóa hiệu suất được thảo luận rất sâu sắc. Dưới đây là những điểm có thể liên hệ với toán học:

1. **Phân chia Batch Size (Batch Size Splitting):** Khi sử dụng `MirroredStrategy`, `fit()` tự động chia `batch_size` cho số lượng bản sao (replica), tức là số lượng GPU. Nếu có N_GPU GPU, và `batch_size` là B , thì mỗi GPU sẽ nhận một mini-batch có kích thước B/N_GPU . Để đảm bảo tất cả các bản sao nhận được các lô có cùng kích thước, B phải chia hết cho N_GPU .
2. **Thuật toán AllReduce:** Đây là một lớp thuật toán phân tán được sử dụng để tổng hợp (ví dụ: tính trung bình, tổng) các giá trị (như gradient) từ nhiều thiết bị và sau đó phân

phối kết quả trở lại tất cả các thiết bị. Mặc dù không có công thức cụ thể, cơ sở của AllReduce liên quan đến các phép toán tuyến tính (tổng, trung bình) trên các vector/tensor.

- **Reduce (ví dụ: sum):** Nếu mỗi thiết bị i có một vector gradient g_i , thì phép toán tổng hợp là $sum_i = 1^{N_{GPU}} g_i$.
- **Broadcast:** Sau khi tổng hợp, kết quả được gửi lại cho tất cả các thiết bị. Các triển khai như NCCL (NVIDIA Collective Communications Library) tối ưu hóa các phép toán này để tận dụng kiến trúc phần cứng đặc biệt của GPU, giảm thiểu thời gian giao tiếp.

3. **Chiến lược Đồng bộ (Synchronous Updates):** Trong MultiWorkerMirroredStrategy, tất cả các worker đều đồng bộ hóa ở bước AllReduce. Điều này có nghĩa là mỗi worker phải đợi cho đến khi tất cả các worker khác đã tính toán gradient của chúng và tham gia vào phép toán AllReduce. Nếu T_{comp} là thời gian tính toán gradient trên một mini-batch và T_{comm} là thời gian giao tiếp (AllReduce), thì tổng thời gian cho một bước là: $T_{step} = T_{comp} + T_{comm}$. Ở đây, $\max(T_{comp}, T_{comm})$ ngụ ý rằng bước này bị giới hạn bởi worker chậm nhất.

4. **Chiến lược Bất đồng bộ (Asynchronous Updates):** Trong ParameterServerStrategy với cập nhật bất đồng bộ, các worker hoạt động độc lập. Khi một worker tính toán xong gradient, nó gửi ngay lập tức đến parameter server để cập nhật. Không có sự chờ đợi giữa các worker. Điều này có thể dẫn đến gradient lỗi thời (stale gradients) vì tham số có thể đã được cập nhật bởi các worker khác.

Văn bản mô tả các phương pháp kỹ thuật để tối ưu hóa việc sử dụng tài nguyên (CPU, GPU, RAM, băng thông) nhằm đạt được hiệu suất cao nhất trong quá trình huấn luyện mô hình học máy quy mô lớn.

Chạy các Tác vụ Huấn luyện Lớn trên Vertex AI

Vertex AI cho phép bạn tạo các tác vụ huấn luyện tùy chỉnh với mã huấn luyện của riêng bạn. Trên thực tế, bạn có thể sử dụng gần như cùng một mã huấn luyện như bạn sẽ sử dụng trên cụm TF của riêng bạn. Điều chính bạn phải thay đổi là nơi chief nên lưu mô hình, các điểm kiểm tra (checkpoints) và nhật ký TensorBoard. Thay vì lưu mô hình vào một thư mục cục bộ, chief phải lưu nó vào GCS, sử dụng đường dẫn được Vertex AI cung cấp trong biến môi trường AIP_MODEL_DIR. Đối với các điểm kiểm tra mô hình và nhật ký TensorBoard, bạn nên sử dụng các đường dẫn trong các biến môi trường AIP_CHECKPOINT_DIR và AIP_TENSORBOARD_LOG_DIR, tương ứng. Tất nhiên, bạn cũng phải đảm bảo rằng dữ liệu huấn luyện có thể được truy cập từ các máy ảo, chẳng hạn như trên GCS, hoặc một dịch vụ GCP khác như BigQuery, hoặc trực tiếp từ web. Cuối cùng, Vertex AI đặt loại tác vụ “chief” một cách rõ ràng, vì vậy bạn nên xác định chief bằng cách sử dụng `resolver.task_type == "chief"` thay vì `resolver.task_id == 0`:

```

import os
import tempfile
from pathlib import Path
import tensorflow as tf

# Giả định các imports khác, tạo MultiWorkerMirroredStrategy và resolver đã đ
ược thực hiện
# Ví dụ:
# strategy = tf.distribute.MultiWorkerMirroredStrategy()
# resolver = tf.distribute.cluster_resolver.TFConfigClusterResolver()
# print(f"Starting task {resolver.task_type} #{resolver.task_id}")

if resolver.task_type == "chief":
    model_dir = os.getenv("AIP_MODEL_DIR") # đường dẫn được cung cấp bởi Vert
ex AI
    tensorboard_log_dir = os.getenv("AIP_TENSORBOARD_LOG_DIR")
    checkpoint_dir = os.getenv("AIP_CHECKPOINT_DIR")
else:
    tmp_dir = Path(tempfile.mkdtemp()) # các worker khác sử dụng thư mục tạm
thời
    model_dir = tmp_dir / "model"
    tensorboard_log_dir = tmp_dir / "logs"
    checkpoint_dir = tmp_dir / "ckpt"

callbacks = [
    tf.keras.callbacks.TensorBoard(log_dir=tensorboard_log_dir),
    tf.keras.callbacks.ModelCheckpoint(filepath=checkpoint_dir)
]
# ... xây dựng và biên dịch bằng cách sử dụng phạm vi chiến lược, giống như t
rước đó ...
# Ví dụ: (X_train, y_train), (X_valid, y_valid) đã được tải và tiền xử lý
# with strategy.scope():
#     model = tf.keras.Sequential([...])
#     model.compile([...])

model.fit(X_train, y_train, validation_data=(X_valid, y_valid), epochs=10,
        callbacks=callbacks)
model.save(model_dir, save_format="tf")

```

Bây giờ bạn có thể tạo một tác vụ huấn luyện tùy chỉnh trên Vertex AI, dựa trên script này. Bạn sẽ cần chỉ định tên tác vụ, đường dẫn đến script huấn luyện của bạn, ảnh Docker để sử dụng cho huấn luyện, ảnh Docker để sử dụng cho dự đoán (sau khi huấn luyện), bất kỳ thư viện Python bổ sung nào bạn có thể cần, và cuối cùng là bucket mà Vertex AI nên sử dụng làm thư mục dàn dựng (staging directory) để lưu trữ script huấn luyện. Theo mặc định, đó cũng là nơi script huấn luyện sẽ lưu mô hình đã huấn luyện, cũng như nhật ký TensorBoard và điểm kiểm tra mô hình (nếu có). Hãy tạo tác vụ:

```

from google.cloud import aiplatform

```

```
# Giả định bucket_name và server_image (đã định nghĩa trước đó)

custom_training_job = aiplatform.CustomTrainingJob(
    display_name="my_custom_training_job",
    script_path="my_vertex_ai_training_task.py", # Đảm bảo tên tệp này khớp v
    ới tệp script của bạn
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-4:latest", # Hoặ
    c phiên bản mới hơn
    model_serving_container_image_uri=server_image,
    requirements=["gcsfs==2022.3.0"], # không cần thiết, đây chỉ là một ví dụ
    staging_bucket=f"gs://{bucket_name}/staging"
)
```

Và bây giờ hãy chạy nó trên hai worker, mỗi worker với hai GPU:

```
# Giả định custom_training_job đã được tạo

mnist_model2 = custom_training_job.run(
    machine_type="n1-standard-4",
    replica_count=2, # Số lượng worker
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2, # Số lượng GPU cho mỗi worker
)
```

Và thế là xong: Vertex AI sẽ cấp phát các nút tính toán bạn yêu cầu (trong hạn ngạch của bạn), và nó sẽ chạy script huấn luyện của bạn trên chúng. Khi tác vụ hoàn tất, phương thức `run()` sẽ trả về một mô hình đã huấn luyện mà bạn có thể sử dụng chính xác như mô hình bạn đã tạo trước đó: bạn có thể triển khai nó đến một điểm cuối, hoặc sử dụng nó để thực hiện dự đoán hàng loạt. Nếu có bất kỳ điều gì không ổn trong quá trình huấn luyện, bạn có thể xem nhật ký trong bảng điều khiển GCP: trong menu điều hướng , chọn Vertex AI → Training, nhấp vào tác vụ huấn luyện của bạn, và nhấp vào VIEW LOGS. Ngoài ra, bạn có thể nhấp vào tab CUSTOM JOBS và sao chép ID tác vụ (ví dụ: 1234), sau đó chọn Logging từ menu điều hướng  và truy vấn `resource.labels.job_id=1234`.

Nếu bạn muốn thử một vài giá trị siêu tham số, một lựa chọn là chạy nhiều tác vụ. Bạn có thể truyền các giá trị siêu tham số vào script của bạn dưới dạng đối số dòng lệnh bằng cách đặt tham số args khi gọi phương thức `run()`, hoặc bạn có thể truyền chúng dưới dạng biến môi trường bằng cách sử dụng tham số `environment_variables`.

Tuy nhiên, nếu bạn muốn chạy một tác vụ điều chỉnh siêu tham số lớn trên đám mây, một lựa chọn tốt hơn nhiều là sử dụng dịch vụ điều chỉnh siêu tham số của Vertex AI. Hãy xem cách thực hiện.

Điều chỉnh Siêu Tham số trên Vertex AI

Dịch vụ điều chỉnh siêu tham số của Vertex AI dựa trên một thuật toán tối ưu hóa Bayes (Bayesian optimization algorithm), có khả năng nhanh chóng tìm ra các kết hợp tối ưu của siêu tham số. Để sử dụng nó, trước tiên bạn cần tạo một script huấn luyện chấp nhận các

giá trị siêu tham số dưới dạng đối số dòng lệnh. Ví dụ, script của bạn có thể sử dụng thư viện chuẩn argparse như sau:

```
import argparse

parser = argparse.ArgumentParser()
parser.add_argument("--n_hidden", type=int, default=2)
parser.add_argument("--n_neurons", type=int, default=256)
parser.add_argument("--learning_rate", type=float, default=1e-2)
parser.add_argument("--optimizer", default="adam")
args = parser.parse_args()
```

Dịch vụ điều chỉnh siêu tham số sẽ gọi script của bạn nhiều lần, mỗi lần với các giá trị siêu tham số khác nhau: mỗi lần chạy được gọi là một **thử nghiệm (trial)**, và tập hợp các thử nghiệm được gọi là một **ngiên cứu (study)**.

Script huấn luyện của bạn sau đó phải sử dụng các giá trị siêu tham số đã cho để xây dựng và biên dịch một mô hình. Bạn có thể sử dụng chiến lược phân phối nhân bản nếu muốn, trong trường hợp mỗi thử nghiệm chạy trên một máy đa GPU. Sau đó, script có thể tải tập dữ liệu và huấn luyện mô hình. Ví dụ:

```
import tensorflow as tf
# Giả định argparse đã được thiết lập và args đã có các giá trị siêu tham số

def build_model(args):
    with tf.distribute.MirroredStrategy().scope():
        model = tf.keras.Sequential()
        model.add(tf.keras.layers.Flatten(input_shape=[28, 28], dtype=tf.uint
8))
        for _ in range(args.n_hidden):
            model.add(tf.keras.layers.Dense(args.n_neurons, activation="relu"
))
        model.add(tf.keras.layers.Dense(10, activation="softmax"))
        opt = tf.keras.optimizers.get(args.optimizer)
        opt.learning_rate = args.learning_rate
        model.compile(loss="sparse_categorical_crossentropy",
                      optimizer=opt,
                      metrics=["accuracy"])
    return model

# ... tải tập dữ liệu (X_train, y_train, X_valid, y_valid) ...
# Ví dụ:
# (X_train, y_train), (X_test, y_test) = tf.keras.datasets.mnist.load_data()
# X_train = X_train.astype('float32') / 255.0
# X_valid = X_valid.astype('float32') / 255.0 # hoặc chia từ X_train
# y_train_sparse = y_train # Nếu sparse_categorical_crossentropy, y_train khô
ng cần one-hot

model = build_model(args)
```

```
history = model.fit(X_train, y_train_sparse, validation_data=(X_valid, y_vali
d_sparse), epochs=10)
```

Cuối cùng, script phải báo cáo hiệu suất của mô hình trở lại dịch vụ điều chỉnh siêu tham số của Vertex AI, để nó có thể quyết định các siêu tham số nào sẽ thử tiếp theo. Để làm điều này, bạn phải sử dụng thư viện `hypertune`, thư viện này được tự động cài đặt trên các máy ảo huấn luyện của Vertex AI:

```
import hypertune
# Giả định history đã có từ model.fit()

hypertune = hypertune.HyperTune()
hypertune.report_hyperparameter_tuning_metric(
    hyperparameter_metric_tag="accuracy", # tên của chỉ số được báo cáo
    metric_value=max(history.history["val_accuracy"]), # giá trị chỉ số
    global_step=model.optimizer.iterations.numpy(),
)
```

Bây giờ script huấn luyện của bạn đã sẵn sàng, bạn cần định nghĩa loại máy bạn muốn chạy nó. Để làm điều này, bạn phải định nghĩa một `CustomJob`, mà Vertex AI sẽ sử dụng làm mẫu cho mỗi thử nghiệm:

```
# Giả định bucket_name đã được định nghĩa
from google.cloud import aiplatform

trial_job = aiplatform.CustomJob.from_local_script(
    display_name="my_search_trial_job",
    script_path="my_vertex_ai_trial.py", # đường dẫn đến script huấn luyện củ
a bạn
    container_uri="gcr.io/cloud-aiplatform/training/tf-gpu.2-4:latest", # Hoặ
c phiên bản mới hơn
    staging_bucket=f"gs://{bucket_name}/staging",
    accelerator_type="NVIDIA_TESLA_K80",
    accelerator_count=2, # trong ví dụ này, mỗi thử nghiệm sẽ có 2 GPU
)
```

Cuối cùng, bạn đã sẵn sàng tạo và chạy tác vụ điều chỉnh siêu tham số:

```
from google.cloud import aiplatform_v1.services.hyperparameter_tuning_job.typ
es.hyperparameter_tuning_job as hpt_pb2
from google.cloud import aiplatform

# Giả định trial_job đã được tạo

hp_job = aiplatform.HyperparameterTuningJob(
    display_name="my_hp_search_job",
    custom_job=trial_job,
    metric_spec={"accuracy": "maximize"}, # Tên chỉ số phải khớp với tên tron
g hypertune.report_hyperparameter_tuning_metric
    parameter_spec={
```

```

        "learning_rate": aiplatform.hyperparameter_tuning.DoubleParameterSpec(
            min=1e-3, max=10, scale="log"),
        "n_neurons": aiplatform.hyperparameter_tuning.IntegerParameterSpec(
            min=1, max=300, scale="linear"),
        "n_hidden": aiplatform.hyperparameter_tuning.IntegerParameterSpec(
            min=1, max=10, scale="linear"),
        "optimizer": aiplatform.hyperparameter_tuning.CategoricalParameterSpec(
            ["sgd", "adam"]),
    },
    max_trial_count=100,
    parallel_trial_count=20,
)
hp_job.run()

```

Ở đây, chúng ta yêu cầu Vertex AI tối đa hóa chỉ số có tên “accuracy”: tên này phải khớp với tên chỉ số được báo cáo bởi script huấn luyện. Chúng ta cũng định nghĩa không gian tìm kiếm, sử dụng thang đo log cho tốc độ học và thang đo tuyến tính (tức là đồng nhất) cho các siêu tham số khác. Tên siêu tham số phải khớp với các đối số dòng lệnh của script huấn luyện. Sau đó, chúng ta đặt số lượng thử nghiệm tối đa là 100, và số lượng thử nghiệm chạy song song tối đa là 20. Nếu bạn tăng số lượng thử nghiệm song song lên (ví dụ) 60, tổng thời gian tìm kiếm sẽ giảm đáng kể, lên đến 3 lần. Nhưng 60 thử nghiệm đầu tiên sẽ được bắt đầu song song, vì vậy chúng sẽ không được hưởng lợi từ phản hồi của các thử nghiệm khác. Do đó, bạn nên tăng số lượng thử nghiệm tối đa để bù đắp — ví dụ, lên khoảng 140.

Điều này sẽ mất khá nhiều thời gian. Khi tác vụ hoàn tất, bạn có thể lấy kết quả thử nghiệm bằng cách sử dụng `hp_job.trials`. Mỗi kết quả thử nghiệm được biểu diễn dưới dạng một đối tượng protobuf, chứa các giá trị siêu tham số và các chỉ số kết quả. Hãy tìm thử nghiệm tốt nhất:

```

import numpy as np

def get_final_metric(trial, metric_id):
    for metric in trial.final_measurement.metrics:
        if metric.metric_id == metric_id:
            return metric.value
    return None # Trả về None nếu không tìm thấy chỉ số

trials = hp_job.trials
trial_accuracies = [get_final_metric(trial, "accuracy")
                    for trial in trials if get_final_metric(trial, "accuracy")
                    is not None] # Lọc các thử nghiệm không có chỉ số

best_trial = None
if trial_accuracies:
    best_trial_index = np.argmax(trial_accuracies)
    best_trial = trials[best_trial_index]
else:
    print("No trials with 'accuracy' metric found.")

```



```

if best_trial:
    print(f">>> max(trial_accuracies) {max(trial_accuracies)}")
    print(f">>> best_trial.id '{best_trial.id}'")
    print(f">>> best_trial.parameters")
    # In parameters
    for param in best_trial.parameters:
        print(f"parameter_id: \"{param.parameter_id}\" value {{ {param.value}
}}")

>>> max(trial_accuracies) 0.977400004863739

>>> best_trial.id '98'

>>> best_trial.parameters
[parameter_id: "learning_rate" value { number_value: 0.001 },
parameter_id: "n_hidden" value { number_value: 8.0 }, parameter_id:
"n_neurons" value { number_value: 216.0 }, parameter_id:
"optimizer" value { string_value: "adam" }
]

```

Vậy là xong! Bây giờ bạn có thể lấy SavedModel của thử nghiệm này, tùy chọn huấn luyện thêm một chút, và triển khai nó vào sản xuất.

Điều chỉnh Siêu Tham số bằng Keras Tuner trên Vertex AI

Thay vì sử dụng dịch vụ điều chỉnh siêu tham số của Vertex AI, bạn có thể sử dụng Keras Tuner (đã giới thiệu trong Chương 10) và chạy nó trên các máy ảo Vertex AI. Keras Tuner cung cấp một cách đơn giản để mở rộng tìm kiếm siêu tham số bằng cách phân phối nó trên nhiều máy: nó chỉ yêu cầu thiết lập ba biến môi trường trên mỗi máy, sau đó chạy mã Keras Tuner thông thường của bạn trên mỗi máy. Bạn có thể sử dụng cùng một script chính xác trên tất cả các máy. Một trong các máy hoạt động như chief (tức là oracle), và các máy khác hoạt động như workers. Mỗi worker hỏi chief xem nên thử những giá trị siêu tham số nào, sau đó worker huấn luyện mô hình bằng cách sử dụng các giá trị siêu tham số này, và cuối cùng nó báo cáo hiệu suất của mô hình trở lại chief, chief sau đó có thể quyết định những giá trị siêu tham số nào worker nên thử tiếp theo.

Ba biến môi trường bạn cần thiết lập trên mỗi máy là:

- **KERASTUNER_TUNER_ID:** Biến này bằng “chief” trên máy chief, hoặc một định danh duy nhất trên mỗi máy worker, chẳng hạn như “worker0”, “worker1”, v.v.
- **KERASTUNER_ORACLE_IP:** Đây là địa chỉ IP hoặc tên máy chủ của máy chief. Chief bản thân nó thường nên sử dụng “0.0.0.0” để lắng nghe trên mọi địa chỉ IP trên máy.
- **KERASTUNER_ORACLE_PORT:** Đây là cổng TCP mà chief sẽ lắng nghe.

Bây giờ bạn đã có tất cả các công cụ và kiến thức cần thiết để tạo các kiến trúc mạng nơ-ron hiện đại và huấn luyện chúng ở quy mô lớn bằng cách sử dụng các chiến lược phân phối khác

n nhau, trên cơ sở hạ tầng của riêng bạn hoặc trên đám mây, và sau đó triển khai chúng ở bất cứ đâu. Nói cách khác, bạn bây giờ có siêu năng lực: hãy sử dụng chúng thật tốt!

Bài tập

1. Một SavedModel chứa gì? Làm thế nào để bạn kiểm tra nội dung của nó?
2. Khi nào bạn nên sử dụng TF Serving? Các tính năng chính của nó là gì? Một số công cụ bạn có thể sử dụng để triển khai nó là gì?
3. Làm thế nào để bạn triển khai một mô hình trên nhiều instance TF Serving?
4. Khi nào bạn nên sử dụng API gRPC thay vì API REST để truy vấn một mô hình được phục vụ bởi TF Serving?
5. Các cách khác nhau mà TFLite giảm kích thước mô hình để nó chạy trên thiết bị di động hoặc thiết bị nhúng là gì?
6. Huấn luyện nhận thức lượng tử hóa (quantization-aware training) là gì, và tại sao bạn lại cần nó?
7. Song song mô hình (model parallelism) và song song dữ liệu (data parallelism) là gì? Tại sao cái sau thường được khuyến nghị?
8. Khi huấn luyện một mô hình trên nhiều máy chủ, bạn có thể sử dụng những chiến lược phân phối nào? Làm thế nào để bạn chọn chiến lược nào để sử dụng?
9. Huấn luyện một mô hình (bất kỳ mô hình nào bạn thích) và triển khai nó lên TF Serving hoặc Google Vertex AI. Viết mã máy khách để truy vấn nó bằng API REST hoặc API gRPC. Cập nhật mô hình và triển khai phiên bản mới. Mã máy khách của bạn bây giờ sẽ truy vấn phiên bản mới. Quay lại phiên bản đầu tiên.
10. Huấn luyện bất kỳ mô hình nào trên nhiều GPU trên cùng một máy bằng cách sử dụng MirroredStrategy (nếu bạn không có quyền truy cập vào GPU, bạn có thể sử dụng Google Colab với thời gian chạy GPU và tạo hai GPU logic). Huấn luyện lại mô hình bằng cách sử dụng CentralStorageStrategy và so sánh thời gian huấn luyện.
11. Tinh chỉnh một mô hình tùy chọn của bạn trên Vertex AI, sử dụng Keras Tuner hoặc dịch vụ điều chỉnh siêu tham số của Vertex AI. Lời giải cho các bài tập này có sẵn ở cuối sổ tay của chương này, tại <https://homl.info/colab3>.

Trước khi chúng ta khép lại chương cuối cùng của cuốn sách này, tôi muốn cảm ơn bạn đã đọc nó đến đoạn cuối cùng. Tôi thực sự hy vọng rằng bạn đã có nhiều niềm vui khi đọc cuốn sách này như tôi khi viết nó, và rằng nó sẽ hữu ích cho các dự án của bạn, dù lớn hay nhỏ.

Nếu bạn tìm thấy lỗi, vui lòng gửi phản hồi. Nói chung, tôi rất muốn biết suy nghĩ của bạn, vì vậy đừng ngần ngại liên hệ với tôi qua O'Reilly, qua dự án GitHub [ageron/hands-on-ml3](#), hoặc trên Twitter tại [@aureliengeron](#).

Trong tương lai, lời khuyên tốt nhất của tôi dành cho bạn là hãy luyện tập và luyện tập: hãy cố gắng hoàn thành tất cả các bài tập (nếu bạn chưa làm), thử nghiệm với các sổ tay, tham gia Kaggle hoặc một số cộng đồng ML khác, xem các khóa học ML, đọc các bài báo, tham dự các hội nghị và gặp gỡ các chuyên gia. Mọi thứ thay đổi nhanh chóng, vì vậy hãy cố gắng cập nhật. Một số kênh YouTube thường xuyên trình bày các bài báo học sâu một cách rất chi tiết,

theo cách dễ tiếp cận. Tôi đặc biệt giới thiệu các kênh của Yannic Kilcher, Letitia Parcalabescu, và Xander Steenbrugge.

Để có những cuộc thảo luận ML hấp dẫn và những hiểu biết cấp cao hơn, hãy nhớ xem ML Street Talk và kênh của Lex Fridman. Việc có một dự án cụ thể để làm việc cũng giúp ích rất nhiều, dù là cho công việc hay để giải trí (lý tưởng là cả hai), vì vậy nếu có bất cứ điều gì bạn luôn mơ ước được xây dựng, hãy thử sức!

Hãy làm việc một cách tăng dần; đừng vội vàng đạt được những điều to lớn ngay lập tức, nhưng hãy tập trung vào dự án của bạn và xây dựng nó từng chút một. Nó sẽ đòi hỏi sự kiên nhẫn và kiên trì, nhưng khi bạn có một con robot đi bộ, hoặc một chatbot hoạt động, hoặc bất cứ thứ gì khác bạn muốn xây dựng, nó sẽ vô cùng bổ ích!

Hy vọng lớn nhất của tôi là cuốn sách này sẽ truyền cảm hứng cho bạn để xây dựng một ứng dụng ML tuyệt vời sẽ mang lại lợi ích cho tất cả chúng ta. Nó sẽ là gì? —Aurélien Géron

Công thức Toán (Phán đoán):

Trong phần này, không có công thức toán học tường minh nào được trình bày. Các khái niệm chủ yếu liên quan đến việc cấu hình môi trường huấn luyện phân tán trên Vertex AI và các nguyên tắc đằng sau việc điều chỉnh siêu tham số.

Tuy nhiên, có một số điểm có thể liên hệ ngầm với toán học hoặc các nguyên tắc kỹ thuật:

12. Chức năng `hypertune.report_hyperparameter_tuning_metric`:

- `metric_value=max(history.history["val_accuracy"])`: Đây là việc lấy giá trị cao nhất của độ chính xác trên tập validation trong suốt quá trình huấn luyện của một thử nghiệm. Hàm `max()` là một phép toán tìm giá trị lớn nhất trong một tập hợp.
- `global_step=model.optimizer.iterations.numpy()`: `global_step` thường là số bước huấn luyện hoặc số lần lặp mà mô hình đã trải qua. Nó đại diện cho “thời điểm” mà chỉ số được báo cáo.

13. Định nghĩa Không gian Tìm kiếm Siêu Tham số:

- `hpt.DoubleParameterSpec(min=1e-3, max=10, scale="log")` cho `learning_rate`: Điều này ngụ ý rằng `learning_rate` sẽ được lấy mẫu từ một phân phối logarit trong khoảng $[10^{-3}, 10]$. Nếu x là một biến được lấy mẫu tuyến tính từ một khoảng, thì 10^x (hoặc e^x) sẽ tạo ra một phân phối logarit. Cụ thể, nếu $u \sim U(0,1)$ là một biến ngẫu nhiên đồng nhất trong khoảng $[0, 1]$, thì một giá trị siêu tham số hp trên thang log trong khoảng $[min_val, max_val]$ có thể được lấy mẫu bằng công thức:
$$\text{base}^{(hp)} = \text{base}^{(min_val)} + u \cdot (\text{base}^{(max_val)} - \text{base}^{(min_val)})$$
 Trong đó $base$ thường là 10 hoặc e .

- `hpt.IntegerParameterSpec(min=1, max=300, scale="linear")` cho `n_neurons` và `n_hidden`: Điều này ngụ ý rằng các số nguyên sẽ được lấy mẫu đồng nhất trong khoảng đã cho.
- `hpt.CategoricalParameterSpec(["sgd", "adam"])` cho `optimizer`: Đây là việc chọn một giá trị từ một tập hợp rời rạc.

14. Số lượng thử nghiệm song song (`parallel_trial_count`) và tổng thời gian tìm kiếm: Văn bản đề cập rằng nếu tăng `parallel_trial_count` từ 20 lên 60, tổng thời gian tìm kiếm có thể giảm 3 lần (a factor of up to 3). Điều này ngụ ý một mối quan hệ nghịch đảo giữa số lượng thử nghiệm song song và thời gian hoàn thành (trong điều kiện lý tưởng): $\text{time} \propto \frac{1}{\text{parallel_trial_count}}$. Tuy nhiên, đây chỉ là một ước tính lý tưởng vì hiệu quả thực tế bị ảnh hưởng bởi các yếu tố như phụ thuộc giữa các thử nghiệm, overhead của hệ thống, và hiệu quả của thuật toán tối ưu hóa Bayes. Các thử nghiệm song song không thể hưởng lợi từ phản hồi của nhau ngay lập tức, do đó cần tăng `max_trial_count` để bù đắp.

Các công thức trên là những suy luận toán học từ các nguyên tắc được mô tả trong văn bản, chứ không phải là công thức tường minh.